

VEIL: Lightweight Zero-Knowledge for Hash-Based Multilinear Proof Systems

Rahul Dalal, Tamir Hemo, Eugene Rabinovich, and Ron D. Rothblum

Succinct*

April 13, 2026

Abstract

As efficient proof systems mature rapidly, more practical use cases that require zero-knowledge (ZK) guarantees are arising. Adding ZK typically requires either composing the non-zk base system with an expensive zk one that proves correctness of the cryptographic hashes performed by the base verifier, or alternatively making tightly coupled modifications to every component of the base protocol.

We introduce VEIL, a lightweight and non-intrusive compiler for hash-based multilinear proof systems. VEIL achieves ZK without these drawbacks. Our approach decouples the protocol’s algebraic interactions from the cryptographic hashing and applies a ZK wrapper solely to the algebraic components. This results in a simple, plausibly post-quantum, protocol that achieves a minimal prover overhead of $(1 + o(1))$, while maintaining the architectural integrity of the base proof system.

Our proof-of-concept implementation demonstrates that, over a 31-bit base prime field, for a trace of 2^{29} field elements, compared to the non-zk proof system, VEIL has a prover overhead of about 3%, verifier overhead of 22% and proof-size overhead of 12%.

*Emails: {rahul,tamir,eugene,ron}@succinct.xyz

Contents

1	Introduction	3
1.1	Our Contribution	3
1.2	Our Approach	4
1.3	Organization	6
2	Preliminaries	6
2.1	Error-Correcting Codes	6
2.2	Interactive Oracle Proofs	7
2.3	Multilinear Interactive Oracle Proofs	8
2.4	Proximity Gaps	9
3	Commit-and-Prove Interactive Oracle Schemes	10
3.1	ZK Lin-Eval PCS	11
3.2	ZK Hadamard-Eval PCS	13
3.3	Zero-Knowledge Circuit Evaluation	18
4	Compiling into a ZK IOP	21
4.1	Intermediate Compilation to Partial ZK	22
4.2	ZK Multilinear PCS	24
4.3	Final Compilation	29
5	Concrete Analysis	30
5.1	End-to-End Abstract Analysis	30
5.2	Concrete Instantiation	31
5.3	Experimental Results	32

1 Introduction

Recent years have seen a rapid advancement in the design of efficient proof-systems. The initial key catalyst for the development of proof-systems was the invention of *zero-knowledge proofs* [GMR89] — these are proof-systems that reveal nothing beyond the correctness of the computation. While the field originated from this seminal work, in recent years much of the focus (in the applied world) has been on developing proof-systems that offer *succinct verification* [BFLS91, Kil92, Mic00] of complex computational statements, but are not necessarily zero-knowledge. As these practical proof-systems mature, and additional use cases arise, there is often a critical need for them to actually offer zero-knowledge (zk) guarantees, so as to protect sensitive witness data.

The “common wisdom” is that converting a succinct proof-system to be zk is not difficult. Indeed, there are a plethora of methods to do so. For example, the classical approach, originating with Ben-Or *et al.* [BGG⁺88], is to compose a base proof-system with an inner zero-knowledge proof system. At a high-level, the prover commits to the non-zk proof and then proves, using an inner zero-knowledge proof-system, that the original verifier would have accepted. This methodology, which arose in the theory literature already in the 80’s, is heavily utilized in practical systems such as Orion [XZS22] as well as many modern zkVMs¹, which often employ a final wrapping phase using a ZK proof system such as Plonk [GWC19] or Groth16 [Gro16]. While conceptually simple and highly modular, this approach can be quite expensive in practice: it forces the inner ZK proof system to prove statements involving complex cryptographic hashing (such as verifying Merkle tree paths). Evaluating these cryptographic hash functions inside an arithmetic circuit introduces massive computational overhead (especially if the field used differs from that of the outer proof-system), thereby bottlenecking the entire system. Additionally, this approach introduces highly undesirable² latency since the inner proof can only start once the base proof is completed.

An alternative approach is to “ZK-ify” each individual component of the proof-system from the ground up — thereby modifying various components such as the proximity tests, sumcheck protocols and other connecting components to natively be zero-knowledge. This approach, considered in a line of work [CFS17, BCF⁺17, BCR⁺19, XZZ⁺19, BCL22, HK24, RW24, Dia25] originally introduced at least a constant prover overhead over the non-zk variant of the protocols, but has been recently highly optimized to achieve minimal (additive) overheads [CFW26]. Still, the main caveat is that this approach is highly intrusive to the code base: it requires surgical changes to every part of the protocol, deeply coupling the ZK logic with the base proof-system. As multilinear proof systems rapidly evolve, maintaining, updating and auditing these tightly coupled, bespoke ZK components becomes an unsustainable engineering burden.

Remark 1.1. *Throughout this work we focus on hash-based proof-systems. We remark that additional techniques can be leveraged when relying on homomorphic cryptographic primitives (e.g., relying on the hardness of discrete log or lattice assumptions). See, e.g., the recent work [KS25]. On the other hand, hash-based systems are post-quantum secure while practical post-quantum homomorphic schemes are comparatively nascent.*

1.1 Our Contribution

We propose a new simple and practical method for compiling a large class of non-zk proof-systems to be zk³. The key benefits of our method are that:

1. It is computationally lightweight (both for the prover and verifier); and

¹A zkVM (zk Virtual Machine) is a succinct argument for a general purpose random-access machine. Despite what the name may suggest, zkVMs are not assumed by default to be zero-knowledge.

²Low latency proving is crucial in some blockchain applications, see e.g. <https://ethproofs.org>.

³Throughout this work, by zero-knowledge we refer to the notion of *honest-verifier* zero-knowledge, in which zero-knowledge is only guaranteed for verifiers that follow the prescribed protocol (or the slightly stronger notion of semi-malicious in which the verifier can choose its coins adversarially). We do so since honest-verifier zero-knowledge suffices for the Fiat-Shamir [FS86] transformation.

2. It is architecturally “non-intrusive”, meaning that zk is added as a “wrapper” over an existing non-zk proof-system, making it far more maintainable.

Thus, we present VEIL, a lightweight compiler for the “ZK-ification” of suitable hash-based multilinear proof-systems. VEIL strikes a balance between the two lines of work mentioned above, achieving modularity close to that of the composition approach [BGG⁺88] but without the high overhead of full recursive proving. We emphasize that we do not aim to achieve optimal asymptotics as in [CFW26] and take a more practical perspective — specifically, we target a multiplicative prover overhead of $(1 + o(1))$ rather than insisting on additive poly-logarithmic overhead, as in [CFW26].

Specifically, we focus on multilinear hash-based proof-systems. We observe that such systems generally follow a common structural pattern:

1. The prover first commits to the multilinear extension of the computation trace.
2. The prover and verifier then engage in an interactive phase that relies purely on field arithmetic. In this phase, all communication between prover and verifier is “in the clear” (i.e., is not mediated by oracles).
3. The protocol ultimately reduces to a multilinear evaluation claim on the trace, which is proven by the verifier relative to the original commitment.

Following the terminology of Kalai and Raz [KR08], we refer to such a protocol as a *multilinear interactive PCP*.

Moreover, we leverage the fact that hash-based multilinear polynomial commitment schemes used in practice typically have the following structure: the data is embedded within a tall matrix M .⁴ The prover encodes each column of this matrix using a Reed-Solomon code and sends an oracle in which each symbol corresponds to a full row of the encoded matrix. For the evaluation proof, the verifier chooses a random linear combination of the columns and then the prover and verifier engage in a proximity proof relative to (a constrained variant of) the code. Examples for practical proximity tests for such constrained Reed-Solomon codes include Basefold [ZCF24], WHIR [ACFY25] and Ligerito [NA25].

Popular multilinear hash-based proof-systems, ranging from the Kalai-Raz [KR08] IPCP to the recent SP1 Hypercube proof-system [SP1], can be captured via this abstraction.

1.2 Our Approach

We propose a lightweight and simple method (which we call VEIL) to compile multilinear proof-systems to be zk. Our approach is inspired by and generalizes an approach proposed recently by Frigo and Shelat⁵ [FS24], which is itself inspired by [AHIV17, WTS⁺18]. In particular, we rely on common techniques that are widely used in zero-knowledge protocols, but, to the best of our knowledge, we utilize them in a manner that is particularly lightweight and easy to maintain.

The main components of VEIL are:

- **Trace Queries:** To protect the raw computation trace, we pad each column with q random field elements before encoding, where q is the query complexity of the proximity test. Intuitively, using the fact that the Reed-Solomon code is a “zk-code” [DGR20, ISVW13], this protects the row openings.

We “hide” the cost of this padding by maintaining the same codeword length n for the Reed-Solomon codewords and only slightly increasing their message sizes. This reduces the distance from δ to $\delta - q/n$ (concrete parameters to bear in mind are $\delta = 1/2$, $q = 2^7$ and $n = 2^{21}$, making this cost negligible).

- **The Proximity Test:** To ensure the final proximity test does not leak information, we append a single additional uniformly random column to the matrix to blind the verifier’s random linear combination.

⁴We follow here the convention of the applied literature while noting that most theory-oriented works consider the transposed case in which the matrix is wide.

⁵The [FS24] protocol is enjoying widespread deployment in Google wallet, see <https://blog.google/products-and-platforms/platforms/google-pay/google-wallet-age-identity-verifications/>.

Note that this means that the underlying proximity test is being run on a random vector and so we do not need for the proximity test itself to be zero-knowledge.

The additional cost here is adding a column to the matrix. Assuming initially the matrix had t columns, the multiplicative prover overhead is roughly $1 + \frac{1}{t}$. In practice a typical setting of t would be $t = 2^7$, and so the overhead is less than 1%.

- **The Interactive Transcript:** With the oracle queries protected by the padding above, we are left only with the interactive phase of the protocol. Crucially, because this phase strictly involves algebraic constraints and does not involve cryptographic hashing, we can utilize the classical Ben-Or et al. approach. Thus, we wrap this specific, lightweight portion of the computation using an “off-the-shelf” zero-knowledge proof system.

In more detail, suppose that in this part of the protocol the prover needs to send a total of c field elements. Then, in our compiled protocol the prover first commits to a random vector $r \in \mathbb{F}^c$. For every $i \in [c]$, instead of sending the i -th symbol τ_i of the transcript, the prover sends $\tau'_i = \tau_i + r_i$. At the end of this phase the prover sends an additional zero-knowledge proof that the commitment to r *could* be opened so that $\tau' - r$ satisfies the base verifier’s decision predicate.

By decoupling the algebraic interaction from the heavy cryptographic commitments, the inner zero-knowledge proof-system that we use will be run on a small and purely algebraic statement (i.e., one not involving any hashing). We further observe that in typical hash-based schemes the communication is very much dominated by the proximity test. Thus, we do not even need for the zero-knowledge proof of the interactive part to be succinct (e.g., the communication can be linear in the base verifier’s decision predicate).

The approach results in a non-intrusive compiler that avoids proving cryptographic hashes in circuits entirely, yielding minimal computational overhead and preserving ease of maintenance of the underlying codebase.

Remark 1.2. *Shielding the proximity test by appending the trace with an extra random column may seem like overkill, but simplifies the protocol considerably.*

Indeed, since the amount of data revealed in the proximity test is much smaller than a full column, adding such a large amount of entropy seems excessive.

Striving for optimal efficiency, one might instead try to modify the proximity test to be zk — indeed, this is the approach taken by [CFW26]. We avoid doing so since this is quite complex and less robust to changes, whereas the additional random column that we add introduces only a very small concrete overhead.

The Inner ZK Proof-System. As noted above, for the “off-the-shelf” zk proof-system we could use any existing zk system. One appealing choice is to use the zk variant of Ligerio [AHIV17], which even offers a partially⁶ succinct proof.

In our implementation we actually construct a new proof-system. The proof-system has a more efficient prover than Ligerio (in the regime we focus on in which most of the computation is linear) but on the other hand is not succinct.

The bird’s eye view is a similar approach to the Ligerio zero-knowledge variant except that we do not embed the data in a matrix but rather keep it as a vector. Doing so lets us avoid using expensive multiplication codes for checking linear claims in the proof (concretely we still use the Reed-Solomon code, but save the cost of the inverse FFT required for the evaluation→evaluation representation used in Ligerio).

Remark 1.3. *Throughout this work, for the sake of simplicity, we rely exclusively on proximity gaps in the unique decoding regime (rather than list decoding bounds).*

For the outer (non-zk) proof-system, adapting the construction to the list decoding regime is straightforward. The inner proof-system requires more care and while we believe it can also be adapted to this regime, we leave this to future work.

⁶The proof size is proportional to the square root of the circuit size.

1.3 Organization

We begin in Section 2 with preliminaries. Then, in Section 3 we present sub-protocols that will be used within our inner zk proof-system. The zk compiler is then presented in Section 4. Finally, we report on our benchmarks in Section 5.

2 Preliminaries

We use $\mathbb{F}$ to denote a finite field and use $\mathbb{F}^\times$ to denote its multiplicative subgroup $\mathbb{F}^\times = \mathbb{F} \setminus \{0\}$.

For $a, b \in \mathbb{F}^n$, the *relative distance* $d_{\text{rel}}(a, b)$ is the fraction of coordinates $i \in [n]$ at which $a_i \neq b_i$. For a (non-empty) subset $S \subseteq \mathbb{F}^n$, we define $d_{\text{rel}}(a, S) := \min_{s \in S} d_{\text{rel}}(a, s)$.

Given vectors $c_1, \dots, c_t \in \mathbb{F}^n$, we denote by $[c_1, \dots, c_t]$ the $n \times t$ matrix whose columns are the c_i . We extend relative distance to matrices row-wise: for $[x_1, \dots, x_t], [x'_1, \dots, x'_t] \in \mathbb{F}^{n \times t}$, we set

$$d_{\text{rel}}([x_1, \dots, x_t], [x'_1, \dots, x'_t])$$

to be the fraction of row indices at which the corresponding rows differ in any coordinate. Similarly, for a non-empty subset $S \subseteq \mathbb{F}^{n \times t}$, we define $d_{\text{rel}}([x_1, \dots, x_t], S)$ to be the minimum of this row-wise distance over all elements of S .

2.1 Error-Correcting Codes

An *error-correcting code* over an alphabet Σ is an injective mapping $C : \Sigma^n \rightarrow \Sigma^m$, for $n, m \in \mathbb{N}$. The *rate* of the code is n/m and its *relative distance* is defined as $\min_{x \neq x'} d_{\text{rel}}(C(x), C(x'))$. The code is *linear* if Σ is a finite field $\mathbb{F}$ and C is a linear function over the field.

2.1.1 Zero-Knowledge Codes

Definition 2.1 (ZK Code [DGR20, ISVW13]). *An error correcting code $C : \mathbb{F}^n \times \mathbb{F}^k \rightarrow \mathbb{F}^m$ is a k -zero-knowledge code if for any set $I \subseteq [m]$ of size k , for any fixed $w \in \mathbb{F}^n$, the projection of $C(w, r)$ onto the coordinates of I is uniformly distributed whenever $r \in \mathbb{F}^k$ is uniformly distributed.*

Remark 2.2. *It can be quickly seen that a linear code $C : \mathbb{F}^n \times \mathbb{F}^k \rightarrow \mathbb{F}^m$ is k -zk if the composition*

$$\mathbb{F}^k \xrightarrow{\iota_2} \mathbb{F}^n \times \mathbb{F}^k \xrightarrow{C} \mathbb{F}^m \xrightarrow{p} \mathbb{F}^k$$

is full-rank whenever ι_2 is the inclusion into the second factor and p is the projection onto some subset of k coordinates.

2.1.2 Multiplicative Codes

Definition 2.3. *Let $a, b \in \mathbb{F}^m$. Then the Hadamard product $a \star b$ is the pointwise product defined by $(a \star b)_i := a_i \cdot b_i$.*

Definition 2.4. *A linear error correcting code $C : \mathbb{F}^n \hookrightarrow \mathbb{F}^m$ is multiplicative if:*

- $\mathbf{1} := (1, \dots, 1) \in C$.
- There is an associated linear product code $C^\star : \mathbb{F}^{n^\star} \hookrightarrow \mathbb{F}^m$ such that for all $v_1, v_2 \in C$, we have $v_1 \star v_2 \in C^\star$.

Remark 2.5. *In the literature, multiplicative codes are usually systematic, which is impossible here because we require codes to be zk. This creates ambiguity in exactly how the message spaces for $C, C^\star$ are represented—our codes really need to be thought of as maps instead of subspaces. The extra condition $\mathbf{1} \in C$ allows us to resolve the relative ambiguity between the representations for C and $C^\star$ as explained below.*

Given such a multiplicative code C , we get a linear map $S := S_C : \mathbb{F}^n \rightarrow \mathbb{F}^{n^*}$ mapping $a \in \mathbb{F}^n$ to the element u such that $C^*(u) = C(a) \star \mathbf{1}$. Furthermore, this satisfies that $C = C^* \circ S$. If $D : \mathbb{F}^{n^*} \rightarrow \mathbb{F}^n$ is a section of S , then this defines a vector product P on $\mathbb{F}^n$ given by $P(a, b) = D(u)$ where $C^*(u) = C(a) \star C(b)$.

Lemma 2.6. *There exists an invertible $r : \mathbb{F}^n \rightarrow \mathbb{F}^n$ such that for the code $C' := C \circ r^{-1}$, there exists a section D of $S_{C'}$ such that for all $a, b \in \mathbb{F}^n$, if u is the element such that $C^*(u) = C(a) \star C(b)$, then $D(u) = a \star b$.*

Proof. This is equivalent to finding D such that the P induced as above is the Hadamard product. There exists a subset I of the coordinates of $\mathbb{F}^m$ such that if p_I is the corresponding coordinate projection on $\mathbb{F}^m$, then the function $w := p_I \circ C$ is an isomorphism. Consider $D_0 := w^{-1} \circ p_I \circ C^*$.

Denote by e_i for $i \in [n]$ the standard coordinate vectors of $\mathbb{F}^n$. Then D_0 induces the Hadamard product in the basis $w^{-1}(e_i)$ for $i \in I$. Therefore, if r is the linear map defined on bases by $w^{-1}(e_i) \mapsto e_i$, we have that $D := r \circ D_0$ suffices for the given r . $\square$

Definition 2.7. *We say a code C is in good form if r being the identity suffices in Lemma 2.6.*

A slightly notation-abusing mnemonic is that D satisfies

$$(D \circ (C^*)^{-1})(C(a) \star C(b)) = a \star b \text{ for all } a, b \in \mathbb{F}^n.$$

Definition 2.8. *Let $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ be a multiplicative code in good form with product code $C^* : \mathbb{F}^{n^*} \rightarrow \mathbb{F}^m$. Call a function D produced from Lemma 2.6 a reduction function for C .*

Examples of Multiplicative Codes. Next, we present two variants of the Reed-Solomon code and show that they are zk-codes.

Example 2.9. *Let $\Xi \subseteq \mathbb{F}$ be a multiplicative subgroup of size 2^m . Let $C : \mathbb{F}^{n+k} \rightarrow \mathbb{F}^{2^m}$ be the coefficient-to-evaluation Reed-Solomon code given by $C((a_i))$ being the evaluations of the degree- $(n+k-1)$ polynomial $a_1 + a_2x + \dots + a_{n+k}x^{n+k-1}$ on Ξ .*

Then C is a k -zk-code (e.g., by Remark 2.2 and the full rank of Vandermonde matrices). Through the FFT algorithm, it may be encoded in time $O(m \cdot 2^m)$.

Example 2.10. *Let $\Xi \subseteq \mathbb{F}$ be a multiplicative subgroup of size 2^m , $\Xi_0 \subseteq \Xi$ a subgroup of size 2^n , and $\zeta_{2^{m+1}} \in \mathbb{F}$ a 2^{m+1} th root of unity. Let $C : \mathbb{F}^{2^n} \rightarrow \mathbb{F}^{2^m}$ be the interpolation Reed-Solomon code such that $C((a_i))$ is computed by interpreting the a_i as the evaluations of a degree- (2^n-1) polynomial on Ξ_0 and then evaluating the polynomial on $\zeta_{2^{m+1}}\Xi$.*

Then C is a multiplicative 2^n -zk-code in good form. Through the FFT algorithm it can be encoded in time $O(m \cdot 2^m)$, though with larger constant factors than coefficient-to-evaluation Reed-Solomon. One choice of product code interpolates evaluations on Ξ_1 to $\zeta_{2^{m+1}}\Xi$, where Ξ_1 is the order- 2^{n+1} subgroup of Ξ . One choice of reduction D is just function restriction which takes time $O(m)$.

2.2 Interactive Oracle Proofs

We next define the notion of *interactive oracle proof*, due to [BCS16, RRR21].

An ℓ -round (public-coin) interactive oracle protocol consists of two entities, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ who interact with each other over ℓ rounds on a common input x . The prover additionally receives an input w called the witness. In each round the prover may send either long oracle messages from which the verifier reads a few of the symbols, or short messages that are read entirely.

The key parameters that we will care about are:

1. **Query Complexity:** the number of bits that the verifier reads from the prover's oracle messages.
2. **Round complexity:** the number of rounds ℓ .
3. **Communication complexity:** the total length of $\mathcal{P}$'s direct messages.

4. **Verifier and Prover complexities:** the computational complexity of the verifier and prover.

Definition 2.11 (Interactive oracle proof (IOP)). *An ℓ -round interactive oracle proof (IOP) with soundness error ε for a relation R is an ℓ -round (public-coin) interactive oracle protocol $(\mathcal{P}, \mathcal{V})$ such that*

- **Completeness:** *If $(x, w) \in R$ then when $\mathcal{V}$ interacts with $\mathcal{P}$, it accepts with probability 1.*
- **Soundness:** *If x is such that for every w it holds $(x, w) \notin R$, then for every prover strategy $\mathcal{P}^*$, when $\mathcal{V}$ interacts with $\mathcal{P}^*$, it accepts with probability at most ε .*

Definition 2.12 (Zero-Knowledge IOP). *An IOP for relation R is zero-knowledge (zk) if there exists a probabilistic polynomial-time simulator Sim such that for every randomness $r \in \{0, 1\}^*$ for $\mathcal{V}$ and $(x, w) \in R$, it holds that $\text{Sim}(r, x)$ is distributed identically to the tuple of all messages received from $\mathcal{P}$ and oracle query responses received by $\mathcal{V}$ when interacting with $\mathcal{P}$ on input (x, w) using randomness r .*

2.3 Multilinear Interactive Oracle Proofs

Our compilation scheme works for any protocol in a class we define of *Multilinear Interactive Oracle Proofs*, or MIOPs.

2.3.1 Multilinear Polynomials

A polynomial $\hat{f} : \mathbb{F}^n \rightarrow \mathbb{F}$ is *multilinear* if it has individual degree at most 1 in each variable. We represent a multilinear polynomial by its vector of evaluations $\hat{f} = (f_i)_i \in \mathbb{F}^{2^n}$ on $\{0, 1\}^n$, where the i -th coordinate corresponds to the element of $\{0, 1\}^n$ representing i in binary — this uniquely determines $\hat{f}$ by the process of multilinear extension:

Define the *equality polynomial* $\text{eq} : \mathbb{F}^n \times \mathbb{F}^n \rightarrow \mathbb{F}$ by

$$\text{eq}(x, z) := \prod_{i=1}^n (x_i z_i + (1 - x_i)(1 - z_i)).$$

This restricts to the indicator function of the diagonal in $(\{0, 1\}^n)^2$; i.e., for $x, y \in \{0, 1\}^n$ we have $\text{eq}(x, y) = 1$ if $x = y$ and 0 otherwise.

The multilinear extension of $\hat{f} : \{0, 1\}^n \rightarrow \mathbb{F}$ can then be given by

$$\hat{F}(z) = \sum_{x \in \{0, 1\}^n} f(x) \cdot \text{eq}(x, z).$$

In particular, $(\text{eq}(x, z_0))_{x \in \{0, 1\}^n}$ gives the coefficients of the linear combination that evaluates any multilinear polynomial at $z_0 \in \mathbb{F}^n$ from its values on $\{0, 1\}^n$.

2.3.2 MIOPs

Definition 2.13. *A Multilinear Interactive Oracle Proof (MIOP) is an IOP that satisfies the following:*

- *All messages sent by $\mathcal{P}$ are either elements of some finite field $\mathbb{F}$ or oracles for evaluations of multilinear polynomials on $\mathbb{F}^n$ for some n .*
- *All messages sent by the verifier are random coin tosses (where the number of messages and their format are fixed beforehand).*
- *The verifier accepts or rejects by, at the end of the interaction, only checking polynomial conditions with inputs that are either prover messages in $\mathbb{F}$ or evaluations of one of the multilinear oracles at some publicly-known point.*

We say that the protocol (P, V) is an MIOP for a relation R with soundness error $2^{-\lambda}$, if the following two conditions hold:

- (Completeness:) if $(x, w) \in R$ then V accepts with probability 1 when interacting with P .
- (Soundness:) if for every w it holds $(x, w) \notin R$, then for any prover strategy P^* , the verifier V accepts when interacting with P^* with probability at most $2^{-\lambda}$.

2.4 Proximity Gaps

We now recall a key property needed for proving soundness of protocols. Given a code $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$, we write C^t for the *interleaved code*: the set of matrices $[c_1, \dots, c_t] \in \mathbb{F}^{m \times t}$ whose columns each lie in the image of C . The distance measure d_{rel} is essentially treating C^t as a (non-linear) code of length m with alphabet $\mathbb{F}^t$.

Definition 2.14 (Proximity Generator). *Let $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ be an error-correcting code, $t \in \mathbb{N}$, and $\delta, \varepsilon_{\text{PG}}(m, |\mathbb{F}|) > 0$. A distribution PG over $\mathbb{F}^t$ is a $(\delta, \varepsilon_{\text{PG}})$ -proximity-generator for C if for all $\delta' \leq \delta$ and all $\mathbf{A} \in \mathbb{F}^{m \times t}$,*

$$d_{\text{rel}}(\mathbf{A}, C^t) \geq \delta' \implies \Pr_{\vec{\rho} \leftarrow \text{PG}} [d_{\text{rel}}(\mathbf{A}\vec{\rho}, C) < \delta'] \leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|).$$

Remark 2.15. *Note also the important companion fact that if $d_{\text{rel}}(\mathbf{A}, C^t) \leq \delta'$, then $d_{\text{rel}}(a, C) \leq \delta'$ for any a in the column span of $\mathbf{A}$.*

We associate two additional parameters to a probability distribution $\text{PG} \sim \mathbb{F}^t$. Its “linear bias” [NN93] and its randomness complexity.

Definition 2.16 (Linear Bias and Randomness Complexity). *Let PG be a distribution over $\mathbb{F}^t$.*

- The (linear) bias of PG is $\varepsilon_{\text{PG}}^\bullet := \max_{\vec{v} \in \mathbb{F}^t \setminus \{0\}} \Pr_{\vec{\rho} \leftarrow \text{PG}} [\vec{\rho} \cdot \vec{v} = 0]$.
- The randomness complexity of PG , denoted $c(\text{PG})$, is the number of independent uniform field elements needed to sample from PG .

Remark 2.17. *Note that if PG is a proximity generator, $\varepsilon_{\text{PG}}^\bullet \leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ by applying the proximity gaps definition to rank-1 matrices $\vec{v}x^{\text{tr}}$.*

Example 2.18. *Some commonly studied PG are:*

1. Linear: $(\rho, 1) \in \mathbb{F}^2$ for ρ uniform over $\mathbb{F}$. This has linear bias $\varepsilon_{\text{PG}}^\bullet = 1/|\mathbb{F}|$.
2. Degree- t : $(\rho^t, \dots, \rho^2, \rho, 1) \in \mathbb{F}^{t+1}$ for ρ uniform over $\mathbb{F}$. This has linear bias $\varepsilon_{\text{PG}}^\bullet = t/|\mathbb{F}|$.
3. t -MLE: If t is a power of 2, the PG is $(\text{eq}(x, z_0))_{x \in \{0,1\}^{\log t}} \in \mathbb{F}^t$ where z_0 is sampled uniformly over $\mathbb{F}^{\log t}$ (see Section 2.3.1). This has linear bias $\varepsilon_{\text{PG}}^\bullet = \log t/|\mathbb{F}|$.

Note that these proximity gaps properties only depend on the image of C , not its method of encoding. Furthermore, if PG is a proximity generator for C then any restriction of PG to some subset of coordinates is also a proximity generator with the same parameters (e.g., by specializing to matrices $\mathbf{A}$ with corresponding all-zero rows).

Example 2.19 ([BCI⁺23, DP24b]). *Let $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ be a Reed-Solomon code. Then it has minimum relative distance $\Delta = 1 - n/m$. It has $(\delta, \varepsilon_{\text{PG}})$ -proximity gaps up to $\Delta/2$ for the following $\text{PG}, \varepsilon_{\text{PG}}$:*

- Degree- t : $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) = mt/|\mathbb{F}|$
- t -MLE: $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) = m \log t/|\mathbb{F}|$

2.4.1 Zero-Knowledge Proximity Generators

To achieve zero-knowledge we will need PG that allow us to add a “mask” with a guaranteed non-zero coefficient.

Definition 2.20. A proximity generator $\text{PG} \sim \mathbb{F}^t$ for some code C is zero-knowledge (zk) if it is supported on $\mathbb{F}^{t-1} \times \mathbb{F}^\times$.

We can modify Example 2.18 to get zk-versions.

Example 2.21. Some zk-PG are:

1. Linear
2. Degree- t
3. augmented t -MLE: If t is a power of 2, the PG is $((\text{eq}(x, z_0))_{x \in \{0,1\}^{\log t}}, 1) \in \mathbb{F}^{t+1}$ where z_0 is sampled uniformly over $\mathbb{F}^{\log t}$. This has linear bias $\varepsilon_{\text{PG}}^\bullet = \log(t)/|\mathbb{F}|$.

It can then be seen that

Lemma 2.22. Let $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ be a Reed-Solomon code. Then it has minimum relative distance $\Delta = 1 - n/m$. It has $(\delta, \varepsilon_{\text{PG}})$ -proximity gaps up to $\Delta/2$ for the following $\text{PG}, \varepsilon_{\text{PG}}$:

- augmented t -MLE: $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) = m \log(t)/|\mathbb{F}|$

3 Commit-and-Prove Interactive Oracle Schemes

In this section, we describe several related commit-and-prove protocols. These will be key components of our compilation step. We start by introducing a general framework for commit-and-prove interactive oracle schemes, which captures the common structure of our protocols.

Definition 3.1 (Commit-and-Prove IOP). Let $\mathcal{W}$ be a witness space, $\mathcal{X}$ be an explicit input space, and $R \subseteq \mathcal{W} \times \mathcal{X}$ be a relation. A commit-and-prove interactive oracle scheme for R consists of three probabilistic polynomial-time algorithms $(\text{Commit}, \text{P}, \text{V})$ with the following semantics:

- $\text{Commit}(w)$: Takes as input a witness $w \in \mathcal{W}$ and outputs an oracle string $\mathfrak{C}$.
- (P, V) : Form an interactive oracle protocol where V gets as input oracle access to $\mathfrak{C}$ (and potentially other oracles sent during the protocol) and explicit access to $x \in \mathcal{X}$, while P gets as input w and x .

We require the following properties:

- (1) (Completeness:) If $(w, x) \in R$, then V accepts with probability 1 when interacting with P and given oracle $\mathfrak{C} = \text{Commit}(w)$.
- (2) (Binding:) For every commitment oracle $\mathfrak{C}$, there exists $w \in \mathcal{W}$ such that for every x with $(w, x) \notin R$ and for every prover P^* , the verifier V rejects with all but $2^{-\lambda}$ probability when interacting with P^* and given inputs $\mathfrak{C}, x$. We call $2^{-\lambda}$ the binding error.

Definition 3.2. We say that a commit-and-prove oracle scheme for R is zero-knowledge if there exists a probabilistic polynomial-time simulator Sim such that for every randomness $r \in \{0,1\}^*$ for V , commitment oracle $\mathfrak{C} = \text{Commit}(w)$, and explicit input x with $(w, x) \in R$, it holds that $\text{Sim}(r, x)$ is distributed identically to the tuple of all messages received from P and oracle query responses received by V when interacting with P on input $\mathfrak{C}, x$ using randomness r .

3.1 ZK Lin-Eval PCS

The first protocol allows one to commit to a vector in $\mathbb{F}^n$ so that later linear function evaluations can be proved about the committed vector with zero-knowledge.

Definition 3.3 (Linear Evaluation Scheme). *A linear evaluation interactive oracle commitment scheme is a commit-and-prove IOP (Definition 3.1) with:*

- *Witness space* $\mathcal{W} = \mathbb{F}^n$,
- *Explicit input space* $\mathcal{X} = \mathbb{F}^n \times \mathbb{F}$ (pairs (ℓ, α)),
- *Relation* $R = \{(w, (\ell, \alpha)) : \langle w, \ell \rangle = \alpha\}$.

Lemma 3.4. *Let $C : \mathbb{F}^{n+k} \rightarrow \mathbb{F}^m$ be a k -zk-code with minimum distance Δ with $\text{PG} \sim \mathbb{F}^2$ a $zk(\delta, \varepsilon_{\text{PG}})$ -proximity-generator for C with $\delta \leq \Delta/2$. Then there exists a zk linear evaluation interactive oracle commitment scheme with the following parameters:*

- *Query Complexity:* k queries to the commitment oracle with an alphabet size of 2 field elements.
- *Binding error:* $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) + (1 - \delta)^k$.
- *Commit Time:* $2T_C(n) + O(n)$, where $T_C(n)$ is the encoding time of C .
- *Prove Time:* $O(n)$.
- *Verify Time:* $T_C(n) + O(n)$
- *Message Complexity:* $n + k + 1$ prover and $c(\text{PG})$ verifier field elements.

Furthermore, this protocol takes the form of an interaction phase in which no queries are made to the commitment oracle followed by a query phase in which k i.i.d. uniform queries are made

3.1.1 Proof of Lemma 3.4

The commitment algorithm and evaluation protocol are described, respectively, in Figs. 1 and 2.

Parameters: k -zk code $C : \mathbb{F}^{n+k} \rightarrow \mathbb{F}^m$

Input: vector $w \in \mathbb{F}^n$

1. P generates a masking vector $r \in \mathbb{F}^n$ uniformly at random.
2. P generates two padding vectors uniformly at random in $\mathbb{F}^k$ and appends them to w, r respectively, producing $w', r' \in \mathbb{F}^{n+k}$.
3. P computes $c = Cw'$ and $c' = Cr'$. It generates and sends the interleaved codeword $\mathfrak{C} \in \mathbb{F}^{2 \times m}$, where $\mathfrak{C}_{0,i} = c_i$ and $\mathfrak{C}_{1,i} = c'_i$, for all $i \in [m]$, as the commitment oracle.

Figure 1: Linear Evaluation Commitment

Parameters: Zk proximity generator $\text{PG} \sim \mathbb{F}^2$ for C
Public Input: commitment $\mathfrak{C} \in \mathbb{F}^{2 \times m}$, linear function $\ell : \mathbb{F}^n \rightarrow \mathbb{F}$, claimed evaluation $\alpha \in \mathbb{F}$.
Prover Private Input: $w', r' \in \mathbb{F}^{n+k}$ from the commitment phase.
Interaction Phase

1. P sends $\beta := \ell(r)$, where r is the n -symbol long prefix of r' .
2. V sends random $(\rho_1, \rho_2) \leftarrow \text{PG}$
3. P sends as an explicit message $x' = \rho_1 \cdot w' + \rho_2 \cdot r'$.
4. V checks that $\ell(x) = \rho_1 \cdot \alpha + \rho_2 \cdot \beta$, where x is the n -symbol prefix of x' .

Query Phase

5. V samples $i_1, \dots, i_k \in [m]$ i.i.d. uniformly at random.
6. For each $j \in [k]$: V queries the i_j -th column of $\mathfrak{C}$ and checks⁷ that $\rho_1 \cdot \mathfrak{C}_{0,i_j} + \rho_2 \cdot \mathfrak{C}_{1,i_j} = (Cx')_{i_j}$.

Figure 2: Linear Evaluation Protocol

Completeness. All checks follow from linearity of ℓ and C : the interaction-phase check gives $\ell(x) = \rho_1 \ell(w) + \rho_2 \ell(r) = \rho_1 \alpha + \rho_2 \beta$, and the query-phase check gives $\rho_1 \mathfrak{C}_{0,i} + \rho_2 \mathfrak{C}_{1,i} = \rho_1 (Cw')_i + \rho_2 (Cr')_i = (Cx')_i$.

Binding. Let $\mathfrak{C} = [\mathfrak{C}_0, \mathfrak{C}_1] \in \mathbb{F}^{2 \times m}$ be an arbitrary commitment oracle. Let $[N'_0, N'_1] \in \mathbb{F}^{2 \times n}$ denote the nearest preimage of $\mathfrak{C}$ under C^2 (breaking ties arbitrarily) and $[N_0, N_1]$ its first n coordinates. Consider the bound witness $w^* := N_0$.

Fix a cheating prover P^* and a false claim (ℓ, α) with $\alpha \neq \langle w^*, \ell \rangle$. Let β be the prover's first message, and let x' be the message sent in response to (ρ_1, ρ_2) (which may depend on (ρ_1, ρ_2)).

Step 5 catches malformed oracles. Either $d_{\text{rel}}([\mathfrak{C}_0, \mathfrak{C}_1], C^2) < \delta$ or not. In the second case, we apply the proximity gaps property (Definition 2.14): $d_{\text{rel}}(\rho_1 \mathfrak{C}_0 + \rho_2 \mathfrak{C}_1, C) \geq \delta$ except with probability at most $\varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ over (ρ_1, ρ_2) . Since $Cx' \in C$, this gives $d_{\text{rel}}(Cx', \rho_1 \mathfrak{C}_0 + \rho_2 \mathfrak{C}_1) \geq \delta$, so all k queries pass with probability at most $(1 - \delta)^k$.

Step 4 catches false evaluations. We may therefore assume that $d_{\text{rel}}([\mathfrak{C}_0, \mathfrak{C}_1], C^2) < \delta$. If the prover sends x' with prefix $x = \rho_1 N_0 + \rho_2 N_1$, then Step 4 requires

$$\rho_1(\ell(N_0) - \alpha) + \rho_2(\ell(N_1) - \beta) = 0.$$

Since $\ell(N_0) \neq \alpha$, the coefficient vector $(\ell(N_0) - \alpha, \ell(N_1) - \beta)$ is nonzero, so the equation holds with probability $\leq \varepsilon_{\text{PG}}^\bullet$.

Step 5 catches inconsistent openings. We may therefore further assume that $x \neq \rho_1 N_0 + \rho_2 N_1$, which implies $x' \neq \rho_1 N'_0 + \rho_2 N'_1$, giving $d_{\text{rel}}(Cx', C(\rho_1 N'_0 + \rho_2 N'_1)) \geq \Delta$ by the minimum distance of C . On the other hand, since $d_{\text{rel}}([\mathfrak{C}_0, \mathfrak{C}_1], C^2) < \delta$, the companion fact gives $d_{\text{rel}}(\rho_1 \mathfrak{C}_0 + \rho_2 \mathfrak{C}_1, \rho_1 CN'_0 + \rho_2 CN'_1) < \delta$. Therefore, by the triangle inequality,

$$d_{\text{rel}}(Cx', \rho_1 \mathfrak{C}_0 + \rho_2 \mathfrak{C}_1) \geq \Delta - \delta \geq \delta.$$

Then each query in Step 5 detects the mismatch with probability at least δ , so all k queries pass with probability at most $(1 - \delta)^k$.

Union bounding within each case and taking a max gives the stated error, using $\varepsilon_{\text{PG}}^\bullet \leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ (Remark 2.17) to absorb the second case into the first.

⁷Depending on the code, this might be optimized by various batched evaluation schemes.

Zero-knowledge. We construct a simulator Sim whose output is distributed identically to the verifier's view in an honest execution. The simulator receives as input the linear function $\ell : \mathbb{F}^n \rightarrow \mathbb{F}$, the claimed evaluation $\alpha \in \mathbb{F}$, and the verifier's randomness: (ρ_1, ρ_2) and the multiset of query indices I from Step 5. It must produce β, x' and oracle query responses $(\mathfrak{C}'_{0,i}, \mathfrak{C}'_{1,i})$ for each unique $i \in I$. We define Sim as follows:

1. Sample $x' \in \mathbb{F}^{n+k}$ uniformly at random. Set $\beta := \rho_2^{-1} \cdot (\ell(x) - \rho_1 \alpha)$, where x is the length- n prefix of x' .
2. For each unique $i \in I$: sample $\mathfrak{C}'_{0,i} \in \mathbb{F}$ uniformly at random and set $\mathfrak{C}'_{1,i} := \rho_2^{-1}((Cx')_i - \rho_1 \mathfrak{C}'_{0,i})$.

By construction, Sim 's output satisfies all of V 's checks. We now show that for any fixed verifier randomness $((\rho_1, \rho_2), I)$, the verifier's view in an honest execution has the same distribution.

Free and determined variables. For each $i \in I$, the query-phase check $\rho_1 \mathfrak{C}_{0,i} + \rho_2 \mathfrak{C}_{1,i} = (Cx')_i$ uniquely determines $\mathfrak{C}_{1,i}$ from $(x', \mathfrak{C}_{0,i})$ (since $\rho_2 \neq 0$ as PG is zk). In an honest execution (where $\ell(w) = \alpha$), the explicit-message check $\ell(x) = \rho_1 \alpha + \rho_2 \beta$ is identically satisfied, so β is determined by x' via $\beta = \rho_2^{-1}(\ell(x) - \rho_1 \alpha)$. Since both Sim and the honest prover compute $(\beta, \mathfrak{C}_{1,i})$ using the same formulas, it suffices to show that the *free variables* $(x', (\mathfrak{C}_{0,i})_{i \in I})$ have the same joint distribution. In Sim , these are mutually independent and uniform.

x' is uniform. In the real protocol, $x' = \rho_1 w' + \rho_2 r'$ where the mask $r' \in \mathbb{F}^{n+k}$ is fully uniform and independent of w' . Since $\rho_2 \neq 0$ (as PG is zk), x' is uniform over $\mathbb{F}^{n+k}$.

x' is independent of $(\mathfrak{C}_{0,i})_{i \in I}$. The commitment entries are $\mathfrak{C}_{0,i} = (Cw')_i$, which depend on the padding $p_w \in \mathbb{F}^k$. We claim p_w is independent of x' : writing $x' = (\rho_1 w + \rho_2 r, \rho_1 p_w + \rho_2 p_r)$, the second component is masked by $\rho_2 p_r$ (uniform, independent of p_w , since $\rho_2 \neq 0$) and the first component depends only on r (independent of p_w). So x' is independent of p_w , and hence of $(Cw')_{i \in I}$.

$(\mathfrak{C}_{0,i})_{i \in I}$ is i.i.d. uniform. Since C is a k -zk-code and $|I| \leq k$, the projection $(Cw')_{i \in I}$ is uniform in $\mathbb{F}^{|I|}$ as p_w varies. Independence from x' (shown above) then gives that $(\mathfrak{C}_{0,i})_{i \in I}$ is i.i.d. uniform and independent of x' , matching Sim .

3.2 ZK Hadamard-Eval PCS

The next protocol allows the sender to commit to three vectors a, b, c and later prove, with zero-knowledge, a Hadamard product relation $a \star b = c$ between them in addition to evaluations $\ell(a), \ell(b), \ell(c)$ at a common linear function ℓ .

Definition 3.5 (Hadamard-Check Scheme). *A Hadamard-check oracle commitment scheme is a commit-and-prove IOP (Definition 3.1) with:*

- *Witness space* $\mathcal{W} = (\mathbb{F}^n)^3$ (triples (a, b, c)),
- *Explicit input space* $\mathcal{X} = \mathbb{F}^n \times \mathbb{F}^3$ (linear functional ℓ and claimed values $(\alpha_a, \alpha_b, \alpha_c)$),
- *Relation* $R = \{((a, b, c), (\ell, (\alpha_*))) : a \star b = c \text{ and } \ell(*) = \alpha_* \text{ for } * \in \{a, b, c\}\}$.

Lemma 3.6. *Let $C : \mathbb{F}^{n+k} \rightarrow \mathbb{F}^m$ be a multiplicative k -zk-code in good form, with minimum relative distance Δ and $\text{PG} \sim \mathbb{F}^4$ a $\text{zk}(\delta, \varepsilon_{\text{PG}})$ -proximity-generator for C with $\delta \leq \Delta/2$. Let C have product code C^* of minimum relative distance Δ^* and $\text{PG}^* \sim \mathbb{F}^2$ a $\text{zk}(\delta^*, \varepsilon_{\text{PG}^*})$ -proximity-generator for C^* with $\delta^* \leq \Delta^*/2$ and $\delta^* \leq \delta$. Let D be the reduction function for C^* (see Definition 2.8). Let T_C, T_{C^*}, T_D be the encoding times of C, C^*, D respectively. Finally, let $\text{LB} \sim \mathbb{F}^n$ have linear bias $\varepsilon_{\text{LB}}^\bullet$.*

Then, there exists a zk Hadamard-check oracle commitment scheme with the following parameters:

- **Query Complexity:** k queries to the commitment oracle with an alphabet size of 5 field elements.
- **Binding error:** $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) + \varepsilon_{\text{PG}^*}(m, |\mathbb{F}|) + \varepsilon_{\text{LB}}^\bullet + (1 - \delta^*)^k$.
- **Commit Time:** $4T_C(n) + T_{C^*}(n) + O(n)$.
- **Prove Time:** $T_{C^*}(n) + T_D(n) + O(n)$.

- **Verify Time:** $T_C(n) + T_{C^*}(n) + T_D(n) + O(n)$.
- **Message Complexity:** $n^* + n + k + 2$ prover and $c(\text{LB}) + c(\text{PG}) + c(\text{PG}^*)$ verifier field elements, where n^* is the message length of C^* .

Furthermore, this protocol takes the form of an interaction phase in which no queries are made to the oracles followed by a query phase in which k i.i.d. uniform queries are made to the commitment oracle.

3.2.1 Proof of Lemma 3.6

The commitment algorithm and evaluation protocol are described, respectively, in Figs. 3 and 4.

Parameters: Multiplicative k -zk code $C : \mathbb{F}^{n+k} \rightarrow \mathbb{F}^m$ in good form with product code C^*

Input: vectors $a, b, c \in \mathbb{F}^n$

1. P generates a masking vector $r_+ \in \mathbb{F}^n$ i.i.d. uniformly at random.
2. P generates a masking vector $r_\times \in \mathbb{F}^{2n}$ uniformly at random.
3. P generates a padding vector in $\mathbb{F}^{2k}$ uniformly at random and appends it to $r_\times$ to produce $r'_\times \in \mathbb{F}^{2(n+k)}$.
4. P generates four padding vectors in $\mathbb{F}^k$ uniformly at random and appends them to $*, r_+$ to produce $*', r'_+ \in \mathbb{F}^{n+k}$ for $* \in \{a, b, c\}$
5. P sends the matrix $\mathfrak{C} \in \mathbb{F}^{\{a,b,c,+, \times\} \times m}$, where $\mathfrak{C}_{*,i} = (C*')_i$ for $* \in \{a, b, c\}$, $\mathfrak{C}_{+,i} = (Cr'_+)_i$, and $\mathfrak{C}_{\times,i} = (C^*r'_\times)_i$.

Figure 3: The Hadamard-Check Commitment

Parameters: Reduction function D for C^* (Definition 2.8), zk proximity generators $\text{PG} \sim \mathbb{F}^4$ for C and $\text{PG}^* \sim \mathbb{F}^2$ for C^* , linear bias generator $\text{LB} \sim \mathbb{F}^n$

Public Input: Commitment $\mathfrak{C} \in \mathbb{F}^{\{a,b,c,+, \times\} \times m}$, linear function $\ell : \mathbb{F}^n \rightarrow \mathbb{F}$, claimed evaluations $\alpha_* \in \mathbb{F}$ for $* \in \{a, b, c\}$.

Prover Private Input: For $* \in \{a, b, c\}$: padded vectors/masks $*', r'_* \in \mathbb{F}^{n+k}$ from commitment scheme phase.

1. V samples a random vector $u \leftarrow \text{LB}$. Use u to also denote the linear function $v \mapsto u \cdot v$.
2. P sends:
 - $\beta := \ell(r_+)$, where r_+ is the length- n prefix of r'_+ ,
 - $\gamma := u(r_{\times}^{\text{red}})$, where $r_{\times}^{\text{red}}$ is the length- n prefix of $D(r'_{\times})$ (recall the reduction function D from Definition 2.8).

3. V sends $(\sigma_1, \sigma_2) \leftarrow \text{PG}^*$.

4. P sends as an explicit message Φ defined to be the input in the product code C^* satisfying

$$C^* \Phi := \sigma_1((Ca') \star (Cb') - Cc') + \sigma_2 \cdot C^* r'_{\times}.$$

5. V sends $(\rho_1, \rho_2, \rho_3, \rho_4) \leftarrow \text{PG}$.

6. P sends as an explicit message $x' := \rho_1 a' + \rho_2 b' + \rho_3 c' + \rho_4 r'_+$.

7. V checks:

- $u(f) = \sigma_2 \cdot \gamma$, where f is the n -symbol prefix of $f' := D(\Phi)$,
- $\ell(x) = \rho_1 \alpha_a + \rho_2 \alpha_b + \rho_3 \alpha_c + \rho_4 \beta$, where x is the length- n prefix of x' .

8. V samples $i_1, \dots, i_k \in [n+k]$ i.i.d. uniformly at random.

9. For each $j \in [k]$: V queries the i_j th column of $\mathfrak{C}$ and checks⁸:

$$\begin{aligned} \sigma_1(\mathfrak{C}_{a,i_j} \mathfrak{C}_{b,i_j} - \mathfrak{C}_{c,i_j}) + \sigma_2 \mathfrak{C}_{\times,i_j} &= (C^* \Phi)_{i_j} \\ \rho_1 \mathfrak{C}_{a,i_j} + \rho_2 \mathfrak{C}_{b,i_j} + \rho_3 \mathfrak{C}_{c,i_j} + \rho_4 \mathfrak{C}_{+,i_j} &= (Cx')_{i_j} \end{aligned}$$

Figure 4: Hadamard-Check Protocol

Completeness. All checks follow from linearity of ℓ and C . The most complicated step is the Hadamard check in Step 7: since D is a reduction function satisfying $D \circ S = \text{id}$, linearity gives $D(\Phi) = \sigma_1(a' \star b' - c') + \sigma_2 \cdot D(r'_{\times})$, whose length- n prefix is $\sigma_1(a \star b - c) + \sigma_2 \cdot r_{\times}^{\text{red}} = \sigma_2 \cdot r_{\times}^{\text{red}}$, so $u(f) = \sigma_2 \gamma$.

Binding. Let $\mathfrak{C} \in \mathbb{F}^{\{a,b,c,+, \times\} \times m}$ be a commitment oracle and define

$$\mathfrak{C}^1 := [\mathfrak{C}_a, \mathfrak{C}_b, \mathfrak{C}_c, \mathfrak{C}_+], \quad \mathfrak{C}^2 := [\mathfrak{C}_a \star \mathfrak{C}_b - \mathfrak{C}_c, \mathfrak{C}_{\times}].$$

Choose

$$(N^1)' := (N'_a, N'_b, N'_c, N'_+) \in \mathbb{F}^{4 \times (n+k)}, \quad (N^2)' := (N'_{abc}, N'_{\times}) \in \mathbb{F}^{2 \times 2(n+k)}$$

⁸Depending on the code, this may be more efficient through a batched evaluation algorithm.

to minimize $d_{\text{rel}}(C(N^1)', \mathfrak{C}^1)$ and $d_{\text{rel}}(C^*(N^2)', \mathfrak{C}^2)$ respectively (breaking ties arbitrarily). Write $*^b := \text{prefix}_n(N'_*)$ for $* \in \{a, b, c\}$, $r_+^b := \text{prefix}_n(N'_+)$, and $r_\times^b := \text{prefix}_n(D(N'_\times))$. We define the bound witness to be (a^b, b^b, c^b) .

Fix a cheating prover P^* and a false claim $(\ell, \alpha_a, \alpha_b, \alpha_c)$. Thus, either (1) $a^b \star b^b \neq c^b$ or (2) $\ell(s^b) \neq \alpha_s$ for some $s \in \{a, b, c\}$. Define

$$T_\times(\vec{\sigma}) := \sigma_1(\mathfrak{C}_a \star \mathfrak{C}_b - \mathfrak{C}_c) + \sigma_2 \mathfrak{C}_\times, \quad T_+(\vec{\rho}) := \rho_1 \mathfrak{C}_a + \rho_2 \mathfrak{C}_b + \rho_3 \mathfrak{C}_c + \rho_4 \mathfrak{C}_+,$$

and their decoded analogues

$$T_\times^C(\vec{\sigma}) := \sigma_1(CN'_a \star CN'_b - CN'_c) + \sigma_2 C^* N'_\times, \quad T_+^C(\vec{\rho}) := \rho_1 CN'_a + \rho_2 CN'_b + \rho_3 CN'_c + \rho_4 CN'_+.$$

Recall that Δ (resp., Δ^*) is the distance of C (resp., C^*) and that $\text{PG} \sim \mathbb{F}^4$ (resp. $\text{PG} \sim \mathbb{F}^2$) is a zk $(\delta, \varepsilon_{\text{PG}})$ -proximity-generator for C with $\delta \leq \Delta/2$ (resp., $(\delta^*, \varepsilon_{\text{PG}^*})$ -proximity-generator for C^* with $\delta^* \leq \Delta^*/2$).

Step 9 catches malformed oracles. Assume that $d_{\text{rel}}(C(N^1)', \mathfrak{C}^1) \geq \delta^*$. By the proximity gaps property of PG (see Definition 2.14) and $\delta^* \leq \delta$ gives that $d_{\text{rel}}(T_+(\vec{\rho}), C) \geq \delta^*$ except with probability $\varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ over $\vec{\rho}$. Since $Cx' \in C$, this gives $d_{\text{rel}}(Cx', T_+(\vec{\rho})) \geq \delta^*$, so the second check of Step 9 accepts with probability at most $(1 - \delta^*)^k$.

Assume that $d_{\text{rel}}(C^*(N^2)', \mathfrak{C}^2) \geq \delta^*$. Since $(N^2)'$ minimized the distance, we also get that $d_{\text{rel}}([CN'_a \star CN'_b - CN'_c, C^* N'_\times], \mathfrak{C}^2) \geq \delta^*$ which gives by the proximity gaps property of PG^* (Definition 2.14) that $d_{\text{rel}}(T_\times(\vec{\sigma}), C^*) \geq \delta^*$ except with probability $\varepsilon_{\text{PG}^*}(m, |\mathbb{F}|)$.

Analyzing the far-case similarly to above, unless $d_{\text{rel}}(C(N^1)', \mathfrak{C}^1), d_{\text{rel}}(C^*(N^2)', \mathfrak{C}^2) < \delta^*$, Step 9 accepts with probability at most $(1 - \delta^*)^k + \varepsilon_{\text{PG}}(m, |\mathbb{F}|) + \varepsilon_{\text{PG}^*}(m, |\mathbb{F}|)$. Therefore we assume that both distances are less than δ^* . In this case, Remark 2.15 gives that

$$\begin{aligned} d_{\text{rel}}(CN'_a \star CN'_b - CN'_c, \mathfrak{C}_a \star \mathfrak{C}_b - \mathfrak{C}_c), d_{\text{rel}}(C^* N'_{abc}, \mathfrak{C}_a \star \mathfrak{C}_b - \mathfrak{C}_c) &< \delta^* \\ \implies d_{\text{rel}}(CN'_a \star CN'_b - CN'_c, C^* N'_{abc}) &< \Delta^*. \end{aligned}$$

By multiplicativity, the last distance is between two C^* -codewords and so we have that:

$$C^* N'_{abc} = CN'_a \star CN'_b - CN'_c.$$

Violation (A): $a^b \star b^b \neq c^b$. Assume we are in the case that $a^b \star b^b \neq c^b$.

Step 7 (first item) catches the false Hadamard claim. Suppose the prover sends Φ such that $C^* \Phi = T_\times^C(\vec{\sigma})$. The reduction-function property (Lemma 2.6) then gives

$$\text{prefix}_n(D(\Phi)) = \sigma_1(a^b \star b^b - c^b) + \sigma_2 r_\times^b.$$

Setting $e := a^b \star b^b - c^b \neq 0$, the check $u(f) = \sigma_2 \gamma$ becomes

$$\sigma_1 u(e) + \sigma_2 (u(r_\times^b) - \gamma) = 0.$$

Since $e \neq 0$ and LB has linear bias $\varepsilon_{\text{LB}}^\bullet$, $\Pr_{u \leftarrow \text{LB}}[u \cdot e = 0] \leq \varepsilon_{\text{LB}}^\bullet$. Conditioned on $u(e) \neq 0$: the coefficient vector $(u(e), u(r_\times^b) - \gamma)$ is nonzero, so the equation holds with probability $\leq \varepsilon_{\text{PG}^*}^\bullet$. Overall the explicit Hadamard check passes with probability at most $\varepsilon_{\text{LB}}^\bullet + \varepsilon_{\text{PG}^*}^\bullet$.

Step 9 (first line) catches inconsistent Φ . We may therefore further restrict to $C^* \Phi \neq T_\times^C(\vec{\sigma})$, giving $d_{\text{rel}}(C^* \Phi, T_\times^C(\vec{\sigma})) \geq \Delta^*$ by the minimum distance of C^* . Since in the case we're considering $d_{\text{rel}}(C^*(N^2)', \mathfrak{C}^2) \leq \delta^*$, Remark 2.15 gives

$$d_{\text{rel}}(T_\times(\vec{\sigma}), T_\times^C(\vec{\sigma})) < \delta^*$$

implying by the triangle inequality that

$$d_{\text{rel}}(C^* \Phi, T_\times(\vec{\sigma})) \geq \Delta^* - \delta^* \geq \delta^*.$$

Then each query detects the mismatch with probability $\geq \delta^*$, so all k queries pass with probability at most $(1 - \delta^*)^k$.

Violation (B): $\ell(s^b) \neq \alpha_s$ for some $s \in \{a, b, c\}$.

Step 7 (second item) catches false linear evaluations. If the prover sends x' such that its prefix $x = \rho_1 a^b + \rho_2 b^b + \rho_3 c^b + \rho_4 r_+^b$, then Step 7 requires

$$\rho_1(\ell(a^b) - \alpha_a) + \rho_2(\ell(b^b) - \alpha_b) + \rho_3(\ell(c^b) - \alpha_c) + \rho_4(\ell(r_+^b) - \beta) = 0.$$

If $\ell(s^b) \neq \alpha_s$ for some $s \in \{a, b, c\}$, the coefficient vector is nonzero, so the equation holds with probability $\leq \varepsilon_{\text{PG}}^*$.

Step 9 (second line) catches inconsistent openings. We may therefore further restrict to $x \neq \rho_1 a^b + \rho_2 b^b + \rho_3 c^b + \rho_4 r_+^b$, where x denotes the length- n prefix of x' , which implies $x' \neq \rho_1 N'_a + \rho_2 N'_b + \rho_3 N'_c + \rho_4 N'_+$, giving $d_{\text{rel}}(Cx', T_+^C(\vec{\rho})) \geq \Delta$ by the minimum distance of C . Since the companion fact gives $d_{\text{rel}}(T_+(\vec{\rho}), T_+^C(\vec{\rho})) < \delta^* \leq \delta$, the triangle inequality gives

$$d_{\text{rel}}(Cx', T_+(\vec{\rho})) \geq \Delta - \delta^* \geq \Delta - \delta \geq \delta.$$

Then each query detects the mismatch with probability $\geq \delta$, so all k queries pass with probability $\leq (1 - \delta)^k$.

Union bounding within each case and taking a max gives the stated error, using $\varepsilon_{\text{PG}^*}^* \leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ and $\varepsilon_{\text{PG}^*}^* \leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ (Remark 2.17) to absorb the linear-bias terms.

Zero-knowledge. We construct a simulator Sim whose output is distributed identically to the verifier's view in an honest execution. The simulator receives as input the linear functional ℓ , the claimed evaluations α_* for $*$ $\in \{a, b, c\}$, and the verifier's randomness: $u, (\rho_1, \rho_2, \rho_3, \rho_4), (\sigma_1, \sigma_2)$, and the multiset of query indices I from Step 8. It must produce the prover's explicit messages β, γ, Φ, x' and oracle query responses $(\mathfrak{C}'_{*,i})_{* \in \{a, b, c, +, \times\}}$ for each unique $i \in I$. We define Sim as follows:

1. Sample $x' \in \mathbb{F}^{n+k}$ uniformly at random and set $\beta := \rho_4^{-1} \cdot (\ell(x) - \rho_1 \alpha_a - \rho_2 \alpha_b - \rho_3 \alpha_c)$, where x denotes the length- n prefix of x' .
2. Sample $\Phi \in \mathbb{F}^{2(n+k)}$ uniformly at random. Set $\gamma := \sigma_2^{-1} \cdot u(f)$, where f is the length- n prefix of $D(\Phi)$.
3. For each unique $i \in I$: sample $\mathfrak{C}'_{a,i}, \mathfrak{C}'_{b,i}, \mathfrak{C}'_{c,i} \in \mathbb{F}$ i.i.d. uniformly at random, then set

$$\begin{aligned} \mathfrak{C}'_{+,i} &:= \rho_4^{-1} ((Cx')_i - \rho_1 \mathfrak{C}'_{a,i} - \rho_2 \mathfrak{C}'_{b,i} - \rho_3 \mathfrak{C}'_{c,i}), \\ \mathfrak{C}'_{\times,i} &:= \sigma_2^{-1} ((C^* \Phi)_i - \sigma_1 \mathfrak{C}'_{a,i} \mathfrak{C}'_{b,i} + \sigma_1 \mathfrak{C}'_{c,i}). \end{aligned}$$

By construction, Sim 's output satisfies all of V 's checks. We now show that for any fixed verifier randomness $(u, \vec{\rho}, \vec{\sigma}, I)$, the verifier's view in an honest execution has the same distribution.

Free and determined variables. For each $i \in I$, the query-phase check equations uniquely determine $(\mathfrak{C}_{+,i}, \mathfrak{C}_{\times,i})$ from $(x', \Phi, \mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})$ (since $\rho_4 \neq 0$ and $\sigma_2 \neq 0$, as PG and PG^* are zk):

$$\begin{aligned} \mathfrak{C}_{+,i} &= \rho_4^{-1} ((Cx')_i - \rho_1 \mathfrak{C}_{a,i} - \rho_2 \mathfrak{C}_{b,i} - \rho_3 \mathfrak{C}_{c,i}), \\ \mathfrak{C}_{\times,i} &= \sigma_2^{-1} ((C^* \Phi)_i - \sigma_1 \mathfrak{C}_{a,i} \mathfrak{C}_{b,i} + \sigma_1 \mathfrak{C}_{c,i}). \end{aligned}$$

Similarly, in an honest execution (where $\ell(*) = \alpha_*$ for $*$ $= a, b, c$ and $a \star b = c$), the explicit-message checks in Step 7 are identically satisfied, so β is determined by x' and γ is determined by Φ :

$$\beta = \rho_4^{-1} (\ell(x) - \rho_1 \alpha_a - \rho_2 \alpha_b - \rho_3 \alpha_c), \quad \gamma = \sigma_2^{-1} \cdot u(\text{prefix}_n(D(\Phi))).$$

(For γ : by linearity of D , $\text{prefix}_n(D(\Phi)) = \sigma_1 \cdot \text{prefix}_n(D(\Phi_0)) + \sigma_2 \cdot r_{\times}^{\text{red}}$ where $\Phi_0 := (C^*)^{-1}((Ca') \star (Cb') - Cc')$. Since $a \star b = c$, the completeness argument gives $\text{prefix}_n(D(\Phi_0)) = 0$, so $u(f) = \sigma_2 \cdot u(r_{\times}^{\text{red}}) = \sigma_2 \gamma$.)

Since both Sim and the honest prover compute $(\mathfrak{C}_{+,i}, \mathfrak{C}_{\times,i})$ and (β, γ) using the same formulas, it suffices to show that the *free variables* $(x', \Phi, (\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})_{i \in I})$ have the same joint distribution. In Sim , these are mutually independent and uniform. We show the real protocol produces the same distribution.

Independence via masking. In the real protocol, the free variables depend on the random inputs as follows:

- $x' = (\rho_1 a' + \rho_2 b' + \rho_3 c') + \rho_4 \cdot r'_+$, where the mask $r'_+ \in \mathbb{F}^{n+k}$ is fully uniform and independent of all other randomness.
- $\Phi = \sigma_1 \Phi_0 + \sigma_2 \cdot r'_\times$, where Φ_0 depends only on (a', b', c') and the mask $r'_\times \in \mathbb{F}^{2(n+k)}$ is fully uniform and independent of all other randomness.
- $(\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i}) = ((Ca')_i, (Cb')_i, (Cc')_i)$, depending only on the padded witnesses (a', b', c') .

We use the following standard fact: if Z is uniform in $\mathbb{F}^d$ and independent of a random variable W , then $g(W) + tZ$ is uniform in $\mathbb{F}^d$ and independent of W , for any function g and any nonzero scalar t . Since r'_+ is independent of $(r'_\times, a', b', c')$ and $\rho_4 \neq 0$ (as PG is zk), the variable x' is uniform over $\mathbb{F}^{n+k}$ and independent of $(\Phi, (\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})_{i \in I})$. By the same reasoning applied to $r'_\times$ (using $\sigma_2 \neq 0$ as PG^{*} is zk), Φ is uniform over $\mathbb{F}^{2(n+k)}$ and independent of $(\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})_{i \in I}$. Therefore x' , Φ , and $(\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})_{i \in I}$ are mutually independent, matching Sim.

Marginal distribution of query responses. It remains to show $(\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})_{i \in I}$ is i.i.d. uniform in $\mathbb{F}^3$. The padding vectors $p_a, p_b, p_c \in \mathbb{F}^k$ (from the commitment, Step 4) are mutually independent and uniform. Since C is a k -zk-code and $|I| \leq k$, for each $* \in \{a, b, c\}$ the projection $(C*_i)_{i \in I}$ is uniform in $\mathbb{F}^{|I|}$. Independence of the three paddings gives that $((Ca')_{i \in I}, (Cb')_{i \in I}, (Cc')_{i \in I})$ is uniform in $\mathbb{F}^{3|I|}$, so the triples $(\mathfrak{C}_{a,i}, \mathfrak{C}_{b,i}, \mathfrak{C}_{c,i})$ are i.i.d. uniform in $\mathbb{F}^3$.

3.3 Zero-Knowledge Circuit Evaluation

We now want a commit-and-prove protocol for arbitrary arithmetic circuit satisfiability. It is well known that the satisfiability of a circuit $\mathcal{C}$ can be reduced to the satisfiability of an R1CS instance checking that $(\mathbf{A}w) \star (\mathbf{B}w) = \mathbf{C}w$ for some $w \in \mathbb{F}^{n'}$ and $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{m' \times n'}$ that can be efficiently constructed from $\mathcal{C}$. We therefore instead construct an R1CS evaluation scheme:

Definition 3.7 (R1CS Evaluation Scheme). *An R1CS interactive oracle commitment scheme is a commit-and-prove IOP (Definition 3.1) with:*

- *Witness space* $\mathcal{W} = \mathbb{F}^{n'}$,
- *Explicit input space:* matrix triples $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ in $\mathcal{X} = (\mathbb{F}^{m' \times n'})^3$,
- *Relation* $R = \{(w, (\mathbf{A}, \mathbf{B}, \mathbf{C})) : (\mathbf{A}w) \star (\mathbf{B}w) = \mathbf{C}w\}$.

We specify the codes used by the sub-protocols. For the zk-linear-eval sub-protocol on vectors of length $n' + 6$: let $C_+ : \mathbb{F}^{n'+6+k} \rightarrow \mathbb{F}^{m_+}$ be a k -zk code with encoding time $T_{C_+}(n')$. For the zk-Hadamard-check sub-protocol on vectors of length $m' + 2$: let $C_\times : \mathbb{F}^{m'+2+k} \rightarrow \mathbb{F}^{m_\times}$ be a multiplicative k -zk code in good form with encoding time $T_{C_\times}(m')$; product code $C_\times^*$ with encoding time $T_{C_\times^*}(m')$; and reduction function D with time $T_D(m')$. Let $\mathbf{LB}_\times \sim \mathbb{F}^{m'+2}$ have linear bias $\varepsilon_{\mathbf{LB}_\times}^\bullet$. Let ε_+ and $\varepsilon_\times$ denote the binding errors of the resulting zk-linear-eval (Lemma 3.4) and zk-Hadamard-check (Lemma 3.6) sub-protocols, and let $c_{V,+}$ and $c_{V,\times}$ denote their verifier message complexities.

We make a technical assumption on $\mathbf{LB}_\times$ (which can be removed if we allow the protocol to be statistically-zk instead of fully zk)

Assumption 3.8. *The last two coordinates (η_1, η_2) of $\mathbf{LB}_\times$ never satisfy $\eta_1 = 0, \eta_2 = 0$, or $\eta_1 + \eta_2 = 0$.*

Lemma 3.9. *Let $C_+, C_\times, C_\times^*$ be codes as specified above (in particular, $C_\times$ in good form). Then there exists a zk R1CS evaluation interactive oracle commitment scheme with the following parameters, where s is the total sparsity of A, B and C (i.e., the total number of nonzero entries):*

- **Query Complexity:** k queries to $\mathfrak{C}_+$ with alphabet size 2 field elements and k queries to $\mathfrak{C}_\times$ with alphabet size 5 field elements.

- Binding error: $3\varepsilon_{\text{LB}_\times}^\bullet + \frac{2}{|\mathbb{F}|} + \max\{\varepsilon_+, \varepsilon_\times\}$.
- Commit Time: $2T_{C_+}(n') + O(n')$.
- Prove Time: $s + 4T_{C_\times}(m') + 2T_{C_\times^*}(m') + T_D(m') + O(m' + n')$.
- Verify Time: $s + T_{C_+}(n') + T_{C_\times}(m') + T_{C_\times^*}(m') + T_D(m') + O(m' + n')$.
- Message Complexity: $n_\times^* + m' + n' + 2k + 14$ prover and $c(\text{LB}_\times) + 1 + c_{V,+} + c_{V,\times}$ verifier field elements, where $n_\times^*$ is the message length of $C_\times^*$.

Furthermore, this protocol takes the form of an interaction phase in which no queries are made to the oracles followed by a query phase in which k i.i.d. uniform queries are made.

3.3.1 Proof of Lemma 3.9

The commitment algorithm and evaluation protocol are described, respectively, in Figs. 5 and 6.

Parameters: zk-linear-eval code C_+ on vectors of length $n' + 6$

Input: witness $w \in \mathbb{F}^{n'}$, R1CS matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{m' \times n'}$

1. P samples $r, s, t \in \mathbb{F}$ i.i.d. uniformly at random. Then
 - (a) P pads w by $(r, s, rs, r - 1, t, (r - 1)t)$ to produce $w' \in \mathbb{F}^{n'+6}$
 - (b) (P, V) pad $\mathbf{A}, \mathbf{B}, \mathbf{C}$ with two rows each realizing the tautological multiplicative relations on the new entries of w' to produce $\mathbf{A}', \mathbf{B}', \mathbf{C}' \in \mathbb{F}^{(m'+2) \times (n'+6)}$.
2. P sends the linear-eval commitment $\mathfrak{C}_+$ of w' .

Figure 5: R1CS Evaluation Commitment

Parameters: Linear bias generator $\text{LB}_\times \sim \mathbb{F}^{m'+2}$; zk-Hadamard-check parameters $(C_\times, C_\times^*, D, \text{PG}_\times, \text{PG}_\times^*, \text{LB})$ on vectors of length $m' + 2$; zk-linear-eval proximity generator PG_+
Public Input: commitment $\mathfrak{C}_+$ from commitment scheme, padded R1CS matrices $\mathbf{A}', \mathbf{B}', \mathbf{C}'$ as defined in commitment.
Prover Private Input: w' and linear-eval intermediate data from commitment scheme.

1. P commits to the Hadamard-check (Fig. 3) oracle $\mathfrak{C}_\times$ for the triple $a := \mathbf{A}'w', b := \mathbf{B}'w', c := \mathbf{C}'w'$.
2. V samples $\ell_\times \leftarrow \text{LB}_\times$. Use $\ell_\times$ to also denote the linear function $v \mapsto \ell_\times \cdot v$.
3. P sends $\alpha_* = \ell_\times(*)$ for $* \in \{a, b, c\}$.
4. V sends $\rho_2 \in \mathbb{F}$ sampled uniformly at random and defines linear functional

$$\ell_+ : v \mapsto \ell_\times(\mathbf{A}'v) + \rho_2 \ell_\times(\mathbf{B}'v) + \rho_2^2 \ell_\times(\mathbf{C}'v).$$

5. (P, V) run zk-Hadamard-Check (Fig. 4) using oracle $\mathfrak{C}_\times$ to prove that $a \star b = c$ and that $\alpha_* = \ell_\times(*)$ for $* = a, b, c$.
6. (P, V) run zk-linear-evaluation (Fig. 2) using oracle $\mathfrak{C}_+$ to check that $\ell_+(w') = \alpha_a + \rho_2 \alpha_b + \rho_2^2 \alpha_c$.

Figure 6: R1CS Evaluation Protocol

Completeness. Suppose $(\mathbf{A}w) \star (\mathbf{B}w) = \mathbf{C}w$. We show that V accepts with probability 1 when interacting with an honest P.

First, the padded R1CS condition $(\mathbf{A}'w') \star (\mathbf{B}'w') = \mathbf{C}'w'$ holds: on the first m' coordinates, this is the original R1CS condition. The two additional rows encode $r \cdot s = rs$ and $(r - 1) \cdot t = (r - 1)t$, which hold by construction of w' .

Let $a = \mathbf{A}'w', b = \mathbf{B}'w', c = \mathbf{C}'w'$. By the above, $a \star b = c$. The prover sends $\alpha_a = \ell_\times(a), \alpha_b = \ell_\times(b), \alpha_c = \ell_\times(c)$. By completeness of zk-Hadamard-Check (Lemma 3.6), the Step-5 check passes with probability 1.

For Step 6, by linearity of $\ell_\times$ and the definition of ℓ_+ :

$$\ell_+(w') = \ell_\times(\mathbf{A}'w') + \rho_2 \ell_\times(\mathbf{B}'w') + \rho_2^2 \ell_\times(\mathbf{C}'w') = \ell_\times(a) + \rho_2 \ell_\times(b) + \rho_2^2 \ell_\times(c) = \alpha_a + \rho_2 \alpha_b + \rho_2^2 \alpha_c.$$

By completeness of zk-linear-evaluation (Lemma 3.4), this check also passes with probability 1.

Binding. Let $w' \in \mathbb{F}^{n'+6}$ be the bound witness of $\mathfrak{C}_+$ under zk-linear-eval (Fig. 2), with $w := \text{prefix}_{n'}(w')$. Let $(a, b, c) \in (\mathbb{F}^{m'+2})^3$ be the bound witness of $\mathfrak{C}_\times$ under zk-Hadamard-check (Fig. 4). The bound witness is w .

Fix a cheating prover P^* with $(\mathbf{A}w) \star (\mathbf{B}w) \neq \mathbf{C}w$. Let $\alpha_a, \alpha_b, \alpha_c$ be the prover's messages in Step 3.

The sub-protocol claims are false. We show that except with probability $3\varepsilon_{\text{LB}_\times}^\bullet + 2/|\mathbb{F}|$ over $\ell_\times, \rho_2$, at least one sub-protocol receives a false claim about its bound witness. Suppose for contradiction that both claims are true: $a \star b = c$ with $\alpha_* = \ell_\times(*)$, and $\ell_+(w') = \alpha_a + \rho_2 \alpha_b + \rho_2^2 \alpha_c$. By definition of ℓ_+ :

$$\ell_\times(\mathbf{A}'w') + \rho_2 \ell_\times(\mathbf{B}'w') + \rho_2^2 \ell_\times(\mathbf{C}'w') = \alpha_a + \rho_2 \alpha_b + \rho_2^2 \alpha_c.$$

If $(\alpha_a, \alpha_b, \alpha_c) \neq (\ell_\times(\mathbf{A}'w'), \ell_\times(\mathbf{B}'w'), \ell_\times(\mathbf{C}'w'))$, this is a nonzero degree-2 polynomial in ρ_2 , vanishing with probability $\leq 2/|\mathbb{F}|$. Therefore consider the case when $\alpha_* = \ell_\times(a) = \ell_\times(\mathbf{A}'w')$, etc. If $a \neq \mathbf{A}'w'$ then $\ell_\times \cdot (a - \mathbf{A}'w')$ is a nonzero linear form in $\ell_\times$, vanishing with probability $\leq \varepsilon_{\text{LB}_\times}^\bullet$; likewise for b, c . Hence

consider the case $(a, b, c) = (\mathbf{A}'w', \mathbf{B}'w', \mathbf{C}'w')$. Combining with $a \star b = c$ yields $(\mathbf{A}'w') \star (\mathbf{B}'w') = \mathbf{C}'w'$. The padding rows are tautologies by construction, so the first m' rows give $(\mathbf{A}w) \star (\mathbf{B}w) = \mathbf{C}w$ which contradicts. A union bound gives non-contradiction probability $3\varepsilon_{\mathbf{L}_{\mathbf{B}_\times}}^\bullet + 2/|\mathbb{F}|$.

Sub-protocols catch the cheater. By the above, at least one sub-protocol receives a false claim. If lin-eval receives a false claim, Step 6 rejects except with the binding error of Lemma 3.4. If Hadamard-check receives a false claim, Step 5 rejects except with the binding error of Lemma 3.6. Taking the minimum and then a union bound gives the claimed binding error.

Zero-knowledge. A valid Sim would be given as input the matrices $\mathbf{A}', \mathbf{B}', \mathbf{C}'$. It is also given the verifier randomness ρ_1, ρ_2 , the zk-lin-eval (Fig. 2) randomness r_+ , and the zk-Hadamard-check (Fig. 4) randomness $r_\times$ (and therefore both ℓ_+ and $\ell_\times$). It needs to generate the α_* for $\star \in \{a, b, c\}$, the zk-lin-eval prover messages M_+ , and the zk-Hadamard-check prover messages $M_\times$. Do so as follows:

1. Generate $\alpha_a, \alpha_b, \alpha_c$ i.i.d. uniformly
2. Generate M_+ using the simulator Sim_+ for zk-lin-eval (Lemma 3.4) on linear functional ℓ_+ , claimed evaluation $\alpha_a + \rho_2\alpha_b + \rho_2^2\alpha_c$, and randomness r_+ .
3. Generate $M_\times$ using the simulator $\text{Sim}_\times$ for zk-Hadamard-check (Lemma 3.6) on linear functional $\ell_\times$, claimed evaluations $\alpha_a, \alpha_b, \alpha_c$, and randomness $r_\times$.

We now check that V's view interacting with an honest P is distributed the same. Let η_1, η_2 be the last two coordinates of $\ell_\times$. Then $\alpha_a, \alpha_b, \alpha_c$ are of the form

$$\alpha_a = \alpha'_a + \eta_1 r + \eta_2(r-1), \quad \alpha_b = \alpha'_b + \eta_1 s + \eta_2 t, \quad \alpha_c = \alpha'_c + \eta_1 r s + \eta_2(r-1)t$$

for some $\alpha'_a, \alpha'_b, \alpha'_c$ and where (η_1, η_2) are the last two coordinates of $\ell_\times$. Then the map

$$(r, s, t) \mapsto (\eta_1 r + \eta_2(r-1), \eta_1 s + \eta_2 t, \eta_1 r s + \eta_2(r-1)t)$$

can be seen to be bijective; if we want to produce image (u, v, w) , we can set

$$r = \frac{u + \eta_2}{\eta_1 + \eta_2}, \quad t = \frac{rv - w}{\eta_2}, \quad s = \frac{v - \eta_2 t}{\eta_1}.$$

using Assumption 3.8 to ensure the denominators are nonzero. Therefore, since r, s, t are i.i.d. random, so are $\alpha_a, \alpha_b, \alpha_c$.

Next, zk-ness for zk-lin-eval and zk-Hadamard-check show that M_+ and $M_\times$ are individually distributed the same as the outputs of the sub-protocols zk-lin-eval and zk-Hadamard-check given any possible values of their inputs.

It remains to show that the $\alpha_\star$ for $\star \in \{a, b, c\}$ and $M_+, M_\times$ are jointly distributed the same as for Sim. This follows since in both the simulator and true protocol 1) zk-lin-eval and zk-Hadamard-check are independent conditioned on the $\alpha_\star$ for $\star \in \{a, b, c\}$ and 2) zk-lin-eval and zk-Hadamard-check are conditionally independent of the $\alpha_\star$ given their inputs.

Remark 3.10. The Ligerio protocol [AHIV17] gives an alternate zero-knowledge arithmetic circuit interactive oracle commitment scheme with a square-root improvement to proof length and verification time over the one presented above. However, in our eventual application to MIOPs, the proof length is usually dominated by oracle commitments and openings instead of the field element messages so the optimization is negligible. This version makes the final compilation scheme easier to implement — see Remark 4.4.

4 Compiling into a ZK IOP

We now describe how to use the above protocols to build our compiler.

4.1 Intermediate Compilation to Partial ZK

This first step is a method producing a fully-zk protocol from a protocol satisfying a certain notion of being “partially zero-knowledge”—intuitively, allowing information to be leaked through some specified subset of prover messages.

Definition 4.1 (Partially Zero-Knowledge Commit-and-Prove Scheme). *A commit-and-prove IOP (Definition 3.1) is partially zero-knowledge if:*

- Messages from P are partitioned into exposed and shielded messages; oracle query responses are always shielded.
- There exists a PPT simulator Sim such that for all verifier randomness $r \in \{0,1\}^*$, commitment $\mathfrak{C} = \text{Commit}(w)$, and input x with $(w, x) \in R$, the output $\text{Sim}(r, x)$ is distributed identically to the shielded prover messages and oracle query responses produced in an honest execution of (P, V) on $\mathfrak{C}, x$ with randomness r .

We also need a notion restricting how the true verifier is allowed to use the exposed prover messages:

Definition 4.2. *A partially-zk IOP with input x is called arithmetically partially zk if V accepts if and only if $\mathcal{C}_{\mathbf{s},x,r}(v) = 0$, where v is the vector of exposed messages sent by P , $\mathbf{s}$ is the vector of shielded messages sent by P and oracle query responses received by V , and $\mathcal{C}_{\mathbf{s},x,r}$ is an arithmetic circuit depending only on $\mathbf{s}$, the input x , and the verifier randomness r (without loss of generality, we assume $\mathcal{C}_{\mathbf{s},x,r}$ is written in the form of an R1CS instance).*

Now, we describe how to compile an arithmetically partially-zk IOP into a fully-zk protocol (later we show how to obtain the partially zk protocol).

Lemma 4.3. *Let $(\mathsf{P}_0, \mathsf{V}_0)$ be an arithmetically partially zk IOP for relation R on inputs x , with prover time P_0 , verifier time V_0 , s exposed messages, and message complexity M_0 . Let $(\text{Commit}_c, \mathsf{P}_c, \mathsf{V}_c)$ be a circuit evaluation interactive oracle scheme, operating on witness vectors of length s , and have commit time $T_{c,\text{Commit}}(n)$, prove time $T_{c,\mathsf{P}}(n)$, verify time $V_c(n)$, and $M_c(n)$ on input circuits of size n .*

Then, there is a fully zk IOP (P, V) for R with the following parameters:

- Oracle Complexity: *that for $(\mathcal{P}_0, \mathcal{V}_0)$ plus that for $(\text{Commit}_c, \mathsf{P}_c, \mathsf{V}_c)$ on circuits of size $s + V_0$.*
- Soundness error: *if $(\text{Commit}_c, \mathsf{P}_c, \mathsf{V}_c)$ and $(\mathsf{P}_0, \mathsf{V}_0)$ have binding error $2^{-\lambda}$, then (P, V) has error $2^{-\lambda+1}$.*
- Prover Time: $P_0 + T_{c,\text{Commit}}(s + V_0) + T_{c,\mathsf{P}}(s + V_0) + O(s + V_0)$.
- Verifier Time: $V_0 + V_c(s + V_0) + O(s + V_0)$.
- Message Complexity: $M_0 + M_c(s + V_0)$.

4.1.1 Proof of Lemma 4.3

The compilation is described in Fig. 7.

Parameters: Arithmetically partially-zk IOP (P_0, V_0) with s exposed messages and verification circuit $C_{s,x,r}$; zk-circuit evaluation scheme $(\text{Commit}_c, P_c, V_c)$.

Public Input: Input x to the original protocol (P_0, V_0) .

1. P generates a uniform random hiding vector $h = (h_i)_i \in \mathbb{F}^s$ and sends the commitment oracle $\text{Commit}_c(h)$.
2. (P, V) run the protocol (P_0, V_0) with the following modification:
 - Whenever P would send the i -th exposed element v_i , it instead sends $v'_i := v_i + h_i$.
3. (P, V) construct⁹ the circuit $C'_{s,x,r,v'}(z) := C_{s,x,r}(v' - z)$
4. (P, V) run the circuit evaluation protocol (P_c, V_c) on the shifted circuit $C'_{s,x,r,v'}$ with respect to commitment $\text{Commit}_c(h)$.

Figure 7: Intermediate Compilation Protocol

Completeness. If $(w, x) \in R$, then an honest execution of (P_0, V_0) satisfies $C_{s,x,r}(v) = 0$. Since $v' - h = v$, we have $C'_{s,x,r,v'}(h) = C_{s,x,r}(v' - h) = C_{s,x,r}(v) = 0$, so completeness of the RICS evaluation scheme implies V accepts.

Soundness. Let P^* be a cheating prover on input x with $(w, x) \notin R$ for all w , and suppose P^* sends commitment $\mathfrak{C}$, exposed messages v' , and shielded messages $\mathbf{s}$. By the binding property of $(\text{Commit}_c, P_c, V_c)$, there exists a value h bound to $\mathfrak{C}$ such that V_c rejects with probability at least $1 - 2^{-\lambda}$ whenever $C'_{s,x,r,v'}(h) \neq 0$.

Now consider the prover P_0^* for (P_0, V_0) that sends exposed messages $v' - h$ and shielded messages $\mathbf{s}$. Since $(w, x) \notin R$ for all w , soundness of (P_0, V_0) gives that V_0 rejects with probability at least $1 - 2^{-\lambda}$, i.e., $C_{s,x,r}(v' - h) \neq 0$. By construction $C_{s,x,r}(v' - h) = C'_{s,x,r,v'}(h)$, so $C'_{s,x,r,v'}(h) \neq 0$, and therefore V_c rejects with probability at least $1 - 2^{-\lambda}$.

A union bound over the two failure events gives that V rejects with probability at least $1 - 2^{-\lambda+1}$.

Zero-knowledge. We construct a simulator Sim as follows:

1. Run Sim_0 (the simulator for the partially-zk protocol (P_0, V_0)) to obtain simulated shielded messages and oracle query responses $\mathbf{s}$.
2. Sample the exposed messages v'_i independently and uniformly at random.
3. Run Sim_c (the simulator for $(\text{Commit}_c, P_c, V_c)$) on the shifted circuit $C'_{s,x,r,v'}$ to generate the circuit evaluation protocol's messages.

We argue that Sim produces a view identically distributed to the real protocol. Fix the verifier randomness r (that for V is the union of that for V_0 and V_c) and consider an honest execution. By definition of Sim_0 , the shielded messages and oracle query responses have the same distribution in the real and simulated executions. In the real protocol, since the hiding entries h_i are i.i.d. uniform and independent of $\mathbf{s}$, the masked messages $v'_i = v_i + h_i$ are also i.i.d. uniform and independent of $\mathbf{s}$ —exactly matching the distribution of the v'_i sampled by Sim . Finally, conditioned on any fixed values of $\mathbf{s}$, v' , x , and verifier randomness, the zero-knowledge

⁹RICS input vectors can without loss of generality be assumed to have final entry 1. In this case, $C'_{s,x,r,v'}$ can be constructed to input vectors of the same length as $C_{s,x,r}$ when considered as an RICS instance

property of Sim_c ensures that the circuit evaluation messages have the same distribution in both executions. Therefore the entire verifier view is identically distributed in the real and simulated protocols.

Remark 4.4 (Practical optimizations). *In our application, the acceptance conditions for $\mathbf{V}$ need not be given as a single arithmetic circuit. Rather, they consist of a long list of affine linear constraints $(\ell_i)_i$ together with a short list of general polynomial constraints on v . The polynomial constraints can be compiled into a small-height R1CS instance determined by $\mathbf{A}, \mathbf{B}, \mathbf{C}$.*

To avoid the overhead of converting between circuit representations, we do not explicitly compute $\mathbf{C}'_{s,r}$ and run a circuit evaluation protocol as in Fig. 7 Items 3 and 4. Instead, we:

1. Use a modified version of the R1CS protocol Fig. 6 to prove the small-height R1CS instance:
 - (a) Rather than setting $a := \mathbf{A}'(v)$, we instead define $a := \mathbf{A}'(v') - \mathbf{A}'(h)$, and similarly for b and c .
 - (b) The internal linear constraint in Item 6 is replaced by $\ell_+(h) = \ell_+(v') - (\alpha_a + \rho_2\alpha_b + \rho_2^2\alpha_c)$
 - (c) Instead of running zk-lin eval in Item 6, we append the above constraint to the ℓ_i .
2. Take a random linear combination of all the ℓ_i to produce a single constraint $\ell(v) = r$. This is verified by proving $\ell(h) = \ell(v') - r$ using the committed hiding vector and zk-dot-product (Fig. 2).

Denote the number of affine linear constraints by n_ℓ and their total sparsity (sum of nonzero coefficients across all ℓ_i) by σ_ℓ . Let m' be the R1CS height and σ_R the total number of nonzero entries in $\mathbf{A}, \mathbf{B}, \mathbf{C}$. Let the sub-protocol codes $C_+, C_\times, C_\times^*, D$ have parameters as in Lemma 3.9, applied to vectors of length $s + 6$ (for zk-lin-eval) and $m' + 2$ (for zk-Hadamard-check). Then the compiled protocol (Lemma 4.3) using this optimization has the following costs:

- Prove Time: $P_0 + \sigma_R + \sigma_\ell + 2T_{C_+}(s) + 4T_{C_\times}(m') + 2T_{C_\times^*}(m') + T_D(m') + O(m' + s)$.
- Verify Time: $V_0 + \sigma_\ell + T_{C_+}(s) + T_{C_\times}(m') + T_{C_\times^*}(m') + T_D(m') + O(m' + s)$.

In the prove time, $2T_{C_+}(s)$ and $4T_{C_\times}(m') + T_{C_\times^*}(m')$ are the costs of the zk-lin-eval and zk-Hadamard-check commitments respectively, σ_R accounts for computing a, b, c via sparse matrix-vector products, and σ_ℓ accounts for evaluating the combined linear constraint $\ell = \sum_i \rho^i \ell_i$. The remaining $O(m' + s)$ terms come from the sub-protocol internals (Lemmas 3.4 and 3.6). The key savings over the generic circuit evaluation (Lemma 4.3) are twofold: the Hadamard-check operates on vectors of length m' (the small R1CS height) rather than s , and the linear constraints are handled via a single dot product of cost $\sigma_\ell + O(s)$ rather than passing through a circuit of size $O(n_\ell \cdot s)$.

4.2 ZK Multilinear PCS

The final protocol commits a multilinear polynomial $\hat{f} : \mathbb{F}^n \rightarrow \mathbb{F}$. We can later use that commitment to give a partially-zk proof that $\hat{f}$ evaluates to a given value at a publicly-known point $x \in \mathbb{F}^n$.

Definition 4.5 (Multilinear Commitment Scheme [DP24a, BFRW25]). *A multilinear polynomial oracle commitment scheme or MCS is a commit-and-prove IOP (Definition 3.1) with:*

- Witness space $\mathcal{W} = \mathbb{F}^{2^n}$ (evaluation vectors of n -variate multilinear polynomials),
- Explicit input space $\mathcal{X} = \mathbb{F}^n \times \mathbb{F}$ (a single evaluation pair (x, y)),
- Relation $R = \{(f, (x, y)) : \hat{f}(x) = y\}$, where $\hat{f}$ is the multilinear polynomial with evaluation vector f ,

together with an auxiliary input space $\mathcal{R}$ and error-correcting code C with minimum distance Δ and message space $\mathcal{W} \times \mathcal{R}$ such that, with probability one, for all $w \in \mathcal{W}$, $\text{Commit}(w) = C(w, r)$ for some $r \in \mathcal{R}$. In addition, we require an extra binding property:

(2') (*C-IOPP Binding*): There exists a constant μ (*IOPP-strength*) such that if $w' = (w, r) \in \mathcal{W} \times \mathcal{R}$ is a witness bound to oracle $\mathfrak{C}$ (as in Item (2)), then the verifier accepts with probability at most $2^{-\mu \cdot d}$ for any $d \leq d_{\text{rel}}(\mathfrak{C}, Cw')$.

Finally, if $\mathcal{R}$ is empty, we call the MCS simple.

Remark 4.6. We need to allow for the extra complication of the auxiliary input $\mathcal{R}$ to achieve zero-knowledge— P needs to mask the original input so that information is not revealed by oracle queries to the commitment.

Recall our conventions from Section 2.3.1 representing an n -variable multilinear polynomial $\hat{f}$ by its evaluation vector $f \in \mathbb{F}^{2^n}$ on $\{0, 1\}^n$.

We build our zk multilinear PCS out of an arbitrary simple PCS for the same code (e.g., in practice we will use the Reed-Solomon code and a simple PCS for it such as [ZCF24, ACFY25, NA25]). For some choices of n and p , let $C : \mathbb{F}^{2^{n-p}+k} \rightarrow \mathbb{F}^{2^m}$ be a k -zk code with minimum relative distance Δ and encoding time $T_C(2^{n-p})$. Let $\text{PG} \sim \mathbb{F}^{2^p+1}$ be a zk $(\delta, \varepsilon_{\text{PG}})$ -proximity-generator for C with $\delta \leq \Delta/2$.

Let $C_0 : \mathbb{F}^{2^{n-p}} \rightarrow \mathbb{F}^{2^m}$ be the restriction of C to the first 2^{n-p} coordinates. Let $(\text{Commit}_0, \mathsf{P}_0, \mathsf{V}_0)$ be a base simple MCS (Definition 4.5) associated to C_0 with binding error $2^{-\lambda_0}$ and IOPP-strength μ_0 , commit time $T_{\text{Commit}_0}(2^{n_0})$, prove time $T_{\mathsf{P}_0}(2^{n_0})$, and verifier time $T_{\mathsf{V}_0}(2^{n_0})$ on n_0 -variable inputs. Finally, let $(\mathsf{P}_0, \mathsf{V}_0)$ involve k_{0,n_0} queries to the oracle generated by Commit_0 .

Lemma 4.7. Let C_0, C and base simple MCS $(\text{Commit}_M, \mathsf{P}_M, \mathsf{V}_M)$ be as specified above. Then there exists an arithmetically partially zk multilinear evaluation interactive oracle commitment scheme $(\mathsf{P}, \mathsf{V}, \text{Commit})$. When applied to n -variable inputs, it depends on a choice of log stacking height $p < n$ and padding $k > k_{0,n}$ and has the following parameters:

- **Associated code:** the interleaved code $C_{\text{int}} : \mathbb{F}^{(2^p+1) \times (2^{n-p}+k)} \rightarrow \mathbb{F}^{(2^p+1) \times 2^m}$ defined by $(C_{\text{int}}M)_{\ell,i} = (CM_{\ell})_i$ where M_{ℓ} is the ℓ th row of M . Here, we interpret $\mathcal{W}$ as the top-right $\mathbb{F}^{2^p \times 2^{n-p}}$ -block and the rest of the coordinates as the auxiliary input $\mathcal{R}$.
- **Binding error:** $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) + \max \{ \varepsilon_{\text{PG}}^{\bullet} + 2^{-\lambda_0}, 2^{-\mu_0 \delta} \}.$
- **IOPP-strength:** $\mu_0(1 - \varepsilon_{\text{PG}}^{\bullet}).$
- **Query Complexity:** $k_{0,n-p}$ queries to $\mathfrak{C}$ with alphabet size $2^p + 1$ field elements (via the virtual oracle $\mathfrak{C}_{\rho}$).
- **Commit Time:** $(2^p + 1) \cdot T_C(2^{n-p} + k) + O(2^n).$
- **Prove Time:** $T_{\mathsf{P}_0}(2^{n-p}) + O(2^n).$
- **Verify Time:** $T_{\mathsf{V}_0}(2^{n-p}) + O(k_{0,n-p} \cdot (k + 2^p)).$
- **Message Complexity:** $2^p + k + 2$ prover and $c(\text{PG})$ verifier field elements, plus the base protocol's messages.

Furthermore, this protocol takes the form of an interaction phase in which no queries are made to the oracles followed by the base protocol's query phase in which $k_{0,n-p}$ queries are made to $\mathfrak{C}$ (via the virtual oracle $\mathfrak{C}_{\rho}$).

4.2.1 Proof of Lemma 4.7

The commitment algorithm and opening protocol are described, respectively, in Figs. 8 and 9. We let $(\text{Commit}_M, \mathsf{P}_M, \mathsf{V}_M)$ be the input MCS based on C_0 .

Completeness. Step 6a passes since $\hat{Y}$ is the multilinear extension of $\ell \mapsto f_{\ell}(x^{(2)})$, so $\hat{Y}(x^{(1)}) = F(x^{(1)}, x^{(2)}) = y$. Step 6b passes since $y_{\ell} = f_{\ell}(x^{(2)})$ so by linearity, $g(x^{(2)}) = \sum_{\ell} \rho_{\ell} y_{\ell} + \rho_{2^p+1} \cdot y_{2^p+1} = a$. Finally, note that $\mathfrak{C}_{\rho} = (Cg') - C(0, g_1) = C_0g$. Then, Step 5 passes by the completeness of $(\mathsf{P}_0, \mathsf{V}_0)$.

Parameters: Variable count n , log stacking height p , k -zk code $C : \mathbb{F}^{2^{n-p}+k} \rightarrow \mathbb{F}^{2^m}$ (and C_0 its restriction to the first 2^{n-p} coordinates)

Input: Multilinear polynomial $F : \mathbb{F}^n \rightarrow \mathbb{F}$ (represented as evaluation vector $f \in \mathbb{F}^{2^n}$)

1. P decomposes f into 2^p vectors f_ℓ , each of length 2^{n-p} and consisting of the coordinates i with their first p binary digits representing ℓ .
2. P pads each f_ℓ with a uniformly random $f_{\ell,1} \in \mathbb{F}^k$ to produce $f'_\ell \in \mathbb{F}^{2^{n-p}+k}$.
3. P generates a uniformly random $f'_{2^p+1} \in \mathbb{F}^{2^{n-p}+k}$.
4. P sends as an oracle the interleaved codeword $\mathfrak{C} \in \mathbb{F}^{(2^p+1) \times 2^m}$ where $\mathfrak{C}_{\ell,i} = (C f'_\ell)_i$

Figure 8: Multilinear Evaluation Commitment

Parameters: Zk proximity generator $\text{PG} \sim \mathbb{F}^{2^p+1}$ for C , base MCS ($\text{Commit}_0, \text{P}_0, \text{V}_0$) for C_0

Public Input: Commitment $\mathfrak{C}$ and parameters from Fig. 8, point $x \in \mathbb{F}^n$, claimed evaluation $y \in \mathbb{F}$.

Prover Private Input: f'_ℓ for $0 \leq \ell \leq 2^p$ from commitment scheme.

As notation, we decompose $x = (x^{(1)} \in \mathbb{F}^p, x^{(2)} \in \mathbb{F}^{n-p})$.

1. P sends as exposed $y_\ell := f_\ell(x^{(2)})$ for $\ell \in \{0, 1\}^p \cup \{2^p + 1\}$.
2. V sends $\vec{\rho} = (\rho_\ell)_{\ell \in \{0, 1\}^p \cup \{2^p + 1\}} \leftarrow \text{PG}$
3. P sends:
 - (a) as shielded, the last k (padding) coordinates g_1 of

$$g' := \sum_{\ell \in \{0, 1\}^p} \rho_\ell f'_\ell + \rho_{2^p+1} \cdot f'_{2^p+1},$$

- (b) as shielded, $a := g(x^{(2)})$, where g is the length- 2^{n-p} prefix of g'

4. P, V define a virtual oracle $\mathfrak{C}_\rho$ for which queries are resolved by querying $\mathfrak{C}$ and computing

$$\mathfrak{C}_{\rho,i} = \sum_{\ell \in \{0, 1\}^p} \rho_\ell \mathfrak{C}_{\ell,i_j} + \rho_{2^p+1} \cdot \mathfrak{C}_{2^p+1,i_j} - (C(0, g_1))_{i_j}.$$

(individually computing each of the values $(C(0, g_1))_{i_j}$ using that g_1 is short).

5. P, V run the base evaluation protocol (P_M, V_M) with the claim $g(x^{(2)}) = a$ and input oracle $\mathfrak{C}_\rho$. All prover messages sent within are shielded. (Note that $\mathfrak{C}_\rho = C_0 g = \text{Commit}_0(g)$).
6. V checks:
 - (a) $\hat{Y}(x^{(1)}) = y$, where $\hat{Y}$ is the multilinear extension of $Y : \{0, 1\}^p \rightarrow \mathbb{F} : \ell \mapsto y_\ell$,
 - (b) $\sum_{\ell \in \{0, 1\}^p} \rho_\ell y_\ell + \rho_{2^p+1} \cdot y_{2^p+1} = a$.

Figure 9: Multilinear Evaluation Opening Protocol

Binding. Let $\mathfrak{C} \in \mathbb{F}^{(2^p+1) \times 2^m}$ be a commitment oracle. Let $[N'_1, \dots, N'_{2^p+1}] \in \mathbb{F}^{(2^p+1) \times (2^{n-p}+k)}$ denote the nearest preimage of $\mathfrak{C}$ under C^{2^p+1} (breaking ties arbitrarily), and let $f_\ell^b := \text{prefix}_{2^{n-p}}(N'_\ell)$. The bound witness is f^b , the evaluation vector obtained by interleaving $(f_\ell^b)_{\ell \in [2^p]}$, with auxiliary data consisting of the remaining components of the N'_ℓ (the padding entries and extra row).

Fix a cheating prover P^* and claim (x, y) . Decompose $x = (x^{(1)}, x^{(2)})$. Let y_ℓ denote the prover's Step 1 messages and a, g_1 the prover's Step 3 messages. Define

$$\mathfrak{C}'_\rho := Cg' = \mathfrak{C}_\rho + C(0, g_1).$$

Evaluation claim binding. We show that if $F^b(x) \neq y$, the verifier accepts with low probability. We show each type of violation is caught by three cases that parallel the argument from zk-linear-eval (Lemma 3.4).

Step 6a forces a lie. If $y_\ell = f_\ell^b(x^{(2)})$ for all $\ell \in [2^p]$, then $\hat{Y}(x^{(1)}) = F^b(x) \neq y$, so the verifier rejects. Hence some $y_\ell \neq f_\ell^b(x^{(2)})$. We may also assume Step 6b passes, i.e., $a = \sum_{\ell \in \{0,1\}^p} \rho_\ell y_\ell + \rho_{2^p+1} \cdot y_{2^p+1}$. Define

$$\begin{aligned} T^C(\vec{\rho}) &:= \sum_{\ell \in \{0,1\}^p} \rho_\ell C N'_\ell + \rho_{2^p+1} \cdot C N'_{2^p+1}, \\ g'^b &:= \sum_{\ell \in \{0,1\}^p} \rho_\ell N'_\ell + \rho_{2^p+1} \cdot N'_{2^p+1}, \end{aligned}$$

with prefix g^b and padding suffix g_1^b , so that $T^C(\vec{\rho}) = Cg'^b$.

The base IOPP catches malformed oracles. Either $d_{\text{rel}}(\mathfrak{C}, C^{2^p+1}) < \delta$ or not. In the second case, the proximity gaps property (Definition 2.14) gives $d_{\text{rel}}(\mathfrak{C}'_\rho, C) \geq \delta$ except with probability $\leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|)$ over $\vec{\rho} \leftarrow \text{PG}$. When the distance bound holds,

$$d_{\text{rel}}(\mathfrak{C}_\rho, C_0) \geq d_{\text{rel}}(\mathfrak{C}_\rho, C) = d_{\text{rel}}(\mathfrak{C}'_\rho, C) \geq \delta.$$

using $C(0, g_1) \in C$ and that C is linear for the equality. Therefore, by the IOPP-strength of the base MCS, the base IOP accepts with probability at most $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) + 2^{-\mu_0\delta}$.

Steps 5–6 catch false evaluations. We may therefore restrict to well-formed oracles: $d_{\text{rel}}(\mathfrak{C}, C^{2^p+1}) < \delta$. Let g_* be the input bound to oracle $\mathfrak{C}_\rho$ under the base IOPP. Consider the case $g_1 = g_1^b$ and $g^b = g_*$. Since some $y_\ell \neq f_\ell^b(x^{(2)})$, the difference $g^b(x^{(2)}) - a$ is a nonzero linear form in the PG coefficients $\vec{\rho}$, so the claim $g_*(x^{(2)}) = g^b(x^{(2)}) = a$ is false except with probability $\leq \varepsilon_{\text{PG}}^\bullet$ over $\vec{\rho}$. When false, the base protocol's evaluation binding gives pass probability $\leq 2^{-\lambda_0}$ for an overall acceptance probability upper bound of $\leq \varepsilon_{\text{PG}}^\bullet + 2^{-\lambda_0}$.

The base IOPP catches inconsistent responses. We may therefore further restrict to $g_1 \neq g_1^b$ or $g^b \neq g_*$. We have $\mathfrak{C}'_\rho - T^C(\vec{\rho}) = \mathfrak{C}_\rho - C(g^b, g_1^b - g_1)$. Since by definition of the N'_ℓ

$$d_{\text{rel}}(\mathfrak{C}, [C N'_1, \dots, C N'_{2^p+1}]) = d_{\text{rel}}(\mathfrak{C}, C^{2^p+1}) < \delta,$$

we have

$$\delta \geq d_{\text{rel}}(\mathfrak{C}'_\rho, T^C(\vec{\rho})) = d_{\text{rel}}(\mathfrak{C}_\rho, C(g^b, g_1^b - g_1)).$$

The definition of minimum distance gives $d_{\text{rel}}(C(g^b, g_1^b - g_1), C_0 g_*) \geq \Delta$, so the reverse triangle inequality gives

$$d_{\text{rel}}(\mathfrak{C}_\rho, C_0 g_*) \geq \Delta - \delta \geq \delta$$

implying acceptance probability at most $2^{-\mu_0\delta}$ by the IOPP-strength of the base MCS.

Overall Bound. The oracle is either malformed or well-formed. In the malformed case, the total chance of acceptance is $\leq \varepsilon_{\text{PG}}(m, |\mathbb{F}|) + 2^{-\mu_0\delta}$. In the well-formed case, taking a maximum over the two subcases gives acceptance probability at most

$$\max \{ \varepsilon_{\text{PG}}^\bullet + 2^{-\lambda_0}, 2^{-\mu_0\delta} \}.$$

Finally, since $\max(A + C, \max(B, C)) \leq A + \max(B, C)$ for $A, B, C \geq 0$, the overall acceptance probability is at most $\varepsilon_{\text{PG}}(m, |\mathbb{F}|) + \max \{ \varepsilon_{\text{PG}}^\bullet + 2^{-\lambda_0}, 2^{-\mu_0\delta} \}$.

Code Proximity Binding. Write $\epsilon := d_{\text{rel}}(\mathfrak{C}, Cw')$ where w' is the bound witness (f^b together with its auxiliary data).

Since the oracle is queried column-wise, ϵ is the fraction of columns i where $\mathfrak{C}_{\ell,i} \neq (CN'_\ell)_i$ for some row ℓ . For each such *bad* column i , the linear form

$$p_i(\vec{\rho}) := \sum_{\ell \in \{0,1\}^p} \rho_\ell (\mathfrak{C}_{\ell,i} - (CN'_\ell)_i) + \rho_{2^p+1} (\mathfrak{C}_{2^p+1,i} - (CN'_{2^p+1})_i)$$

is nonzero in the PG coefficients, so $\Pr_{\vec{\rho} \leftarrow \text{PG}}[p_i(\vec{\rho}) = 0] \leq \epsilon_{\text{PG}}^\bullet$. Since $T(\vec{\rho})_i - T^C(\vec{\rho})_i = p_i(\vec{\rho})$, linearity of expectation gives

$$\mathbb{E}_{\vec{\rho} \leftarrow \text{PG}}[d_{\text{rel}}(T(\vec{\rho}), T^C(\vec{\rho}))] \geq \epsilon(1 - \epsilon_{\text{PG}}^\bullet) =: \delta'.$$

In particular, there exists $\vec{\rho}_0$ in the support of PG with $d_{\text{rel}}(T(\vec{\rho}_0), T^C(\vec{\rho}_0)) \geq \delta'$.

Fix any $\epsilon_0 < \min(\delta', \delta)$. We claim $d_{\text{rel}}(T(\vec{\rho}_0), C) > \epsilon_0$. If not, then $d_{\text{rel}}(T(\vec{\rho}_0), C) \leq \epsilon_0 < \delta$, so the nearest codeword c satisfies $d_{\text{rel}}(T(\vec{\rho}_0), c) \leq \epsilon_0$. If $c = T^C(\vec{\rho}_0)$, then $d_{\text{rel}}(T(\vec{\rho}_0), T^C(\vec{\rho}_0)) \leq \epsilon_0 < \delta'$, contradicting the choice of $\vec{\rho}_0$. If $c \neq T^C(\vec{\rho}_0)$, the triangle inequality gives $\Delta \leq d_{\text{rel}}(c, T^C(\vec{\rho}_0)) \leq \epsilon_0 + d_{\text{rel}}(T(\vec{\rho}_0), T^C(\vec{\rho}_0)) \leq \epsilon_0 + \epsilon$; but $\epsilon_0 < \delta' = \epsilon(1 - \epsilon_{\text{PG}}^\bullet)$, so $\epsilon_0 + \epsilon < \epsilon(2 - \epsilon_{\text{PG}}^\bullet) \leq \Delta$ whenever $\epsilon \leq \delta$, a contradiction. (For $\epsilon > \delta$, see below.)

Therefore, applying the proximity gaps property (Definition 2.14) gives

$$\Pr_{\vec{\rho} \leftarrow \text{PG}}[d_{\text{rel}}(T(\vec{\rho}), C) \leq \epsilon_0] \leq \epsilon_{\text{PG}}(m, |\mathbb{F}|).$$

When $d_{\text{rel}}(T(\vec{\rho}), C) > \epsilon_0$, the same $\alpha + \beta$ tradeoff as in the malformed-oracles case (with ϵ_0 in place of δ) gives combined accept probability $\leq 2^{-\mu_0 \epsilon_0}$. Taking $\epsilon_0 \rightarrow \min(\delta', \delta)$, the overall accept probability is at most

$$\epsilon_{\text{PG}}(m, |\mathbb{F}|) + 2^{-\mu_0 \cdot \min(\delta', \delta)}.$$

For $\epsilon > \delta$: the same argument applies with $\epsilon_0 < \delta$; the unique-decoding contradiction $c \neq T^C(\vec{\rho}_0)$ may no longer close, but one can verify that $\epsilon_0 + \epsilon < \Delta$ still holds whenever $\epsilon < \Delta/(2 - \epsilon_{\text{PG}}^\bullet)$, which covers a neighbourhood above δ . In any case the bound above with $\min(\delta', \delta) = \delta$ gives IOPP-strength $\mu_0(1 - \epsilon_{\text{PG}}^\bullet)$.

Partial Zero-knowledge. A valid Sim gets as input (x, y) and the verifier randomness: $\vec{\rho}$ and the randomness ρ_b used in the base protocol. It has to generate all prover messages in Step 3 and the base evaluation protocol in Step 5. In the true base evaluation protocol, prover messages come in two forms: either oracle queries $(\mathfrak{C}_{\rho, j_i})_i$ to $\mathfrak{C}_\rho$, or messages μ_s that only depend on g . In the true Step 5, the verifier instead sees the $(\mathfrak{C}_{\ell, j_i})_{\ell, i}$ and the μ_s .

We define Sim as follows:

1. Generate a $g' \in \mathbb{F}^{2^{n-p}+k}$ uniformly at random.
2. For each i generate the oracle queries $\mathfrak{C}_{\ell, j_i}$ for $\ell \in \{0, 1\}^p$ uniformly at random and set

$$\mathfrak{C}_{2^p+1, j_i} = \rho_{2^p+1}^{-1} \left((Cg')_{j_i} - \sum_{\ell \in \{0, 1\}^p} \rho_\ell \mathfrak{C}_{\ell, j_i} \right)$$

3. Steps 3–5 form a subprotocol that only requires x, g' as prover inputs and $x, g_1, (\mathfrak{C}_{\ell, j_i})_{\ell, i}$ as verifier inputs. Run both sides of this subprotocol to generate all non- $(\mathfrak{C}_{\ell, i})_{\ell, i}$ messages in Steps 3–5.

All outputs produced by Sim and in the true protocol with an honest prover only depend on public inputs, verifier randomness, the $(\mathfrak{C}_\rho)_{j_i}$, and g' . It therefore suffices to show that g' and $(\mathfrak{C}_\rho)_{j_i}$ have the same distribution in Sim and the true protocol.

First, since f'_{2^p+1} is uniformly random and $\rho_{2^p+1} \neq 0$ (as PG is zk), the term $\rho_{2^p+1} \cdot f'_{2^p+1}$ is uniformly random implying g' is uniform as well.

Next, since the $f_{\ell, 1}$ for $\ell \in \{0, 1\}^p \cup \{2^p + 1\}$ are i.i.d. uniform, $k > k_{0,n}$ and C is a k -zk-code, the $\mathfrak{C}_{\ell, j_i}$ are i.i.d. uniform and independent of g . In addition, since $f_{2^p+1, 1}$ is uniform and $\rho_{2^p+1} \neq 0$, g_1 is uniform and independent of g and the $\mathfrak{C}_{\ell, j_i}$. In total, these $\mathfrak{C}_{\ell, j_i}$ are i.i.d. uniform and independent of g' , which suffices since this determines the final $\mathfrak{C}_{2^p+1, j_i}$ in both Sim and the true protocol.

Arithmetic: The verifier checks are in step 6 and the base protocol. The former are linear expressions in shielded and exposed messages and the latter only involve shielded messages and public inputs.

4.3 Final Compilation

Our final compilation combines Lemma 4.3 with the following:

Lemma 4.8. *Let (P_0, V_0) be an MIOP for relation R on inputs x with soundness error ϵ_0 . In addition, let $(\text{Commit}_{\text{ml}}, P_{\text{ml}}, V_{\text{ml}})$ be an arithmetically partially zk multilinear evaluation interactive oracle commitment scheme with binding error ϵ_{bind} . Then the protocol (P, V) described in Fig. 10 is an arithmetically partially zk IOP for relation R with soundness error at most $\epsilon_0 + \epsilon_{\text{bind}}$.*

4.3.1 Proof of Lemma 4.8

Without loss of generality, we may assume that the input MIOP (P_0, V_0) makes at most one evaluation query to each multilinear oracle — see [RR24] for a standard technique to convert MIOPs into this form. The compilation is then described in Fig. 10.

Parameters: MIOP (P_0, V_0) ; zk-Multilinear-PCS $(\text{Commit}_M, P_M, V_M)$.
Public Input: Input x to the original protocol (P_0, V_0) .

1. (P, V) run the protocol (P_0, V_0) with the following modifications:
 - If P would send a field element $\alpha_i \in \mathbb{F}$, it sends it exposed.
 - If P would send a multilinear oracle f , it instead sends the zk-Multilinear-PCS (Fig. 8) commitment oracle $\text{Commit}_M(f)$.
 - If V would query a multilinear oracle f at point z to receive y , P instead sends y exposed.
 - At the end, for each multilinear commitment, (P, V) runs the (P_M, V_M) proof protocol on its evaluation claim.

Figure 10: Final Compilation: MIOP to Arithmetically Partially-ZK IOP

Completeness. The protocol consists of two steps: first running the original protocol. Then doing the end checks of the zk-Multilinear-PCS (Lemma 4.7). If $(w, x) \in R$, then, with an honest P the first step is accepted because (P_0, V_0) is complete. The second step is accepted since the zk-Multilinear-PCS is complete.

Soundness. Assume no w satisfies $(w, x) \in R$, and fix an arbitrary prover P^* . For each commitment oracle $\mathcal{C}_j$ sent by P^* , let f_j^b be the bound witness under the PCS. We partition the event that the compiled verifier accepts into two cases.

All evaluation claims consistent. Suppose every claimed evaluation y_j equals $f_j^b(z_j)$. Then the MIOP verifier's view is identical to an execution with oracles f_j^b . Since no w satisfies $(w, x) \in R$, the MIOP soundness gives acceptance probability $\leq \epsilon_0$.

Some evaluation claim inconsistent. Suppose $y_j \neq f_j^b(z_j)$ for some j . Since the compiled verifier accepts only if all PCS proofs pass, and in particular the PCS proof for commitment j must pass with a false evaluation claim, the PCS binding gives acceptance probability $\leq \epsilon_{\text{bind}}$.

Since these two cases are exhaustive, the overall acceptance probability is at most $\epsilon_0 + \epsilon_{\text{bind}}$.

Partial Zero-knowledge. The only shielded messages are sent within the zk-Multilinear-PCS (Lemma 4.7) openings. There is exactly one opening per commitment, and these are independent, so the simulators generating the shielded messages for each may be composed.

Finally, all oracle queries only happen within the zk-Multilinear-PCS where they are shielded.

Arithmetic The only checks the verifier runs come from the original MIOP or the internal zk-Multilinear-PCS openings. These are all polynomial.

5 Concrete Analysis

The full compilation combines the final compilation (Lemma 4.8) with the intermediate compilation (Lemma 4.3). This section analyzes the resulting overhead: first in terms of abstract sub-protocol parameters (Section 5.1), then with concrete code and proximity generator choices (Section 5.2).

For simplicity, suppose the MIOP sends a single multilinear oracle with n variables, stacking height p , and padding k . The case of multiple oracles is analogous, with the PCS contributions replaced by sums over the individual oracles. We assume the base MIOP uses a “stacked” variant of the base simple MCS ($\text{Commit}_M, P_M, V_M$)—a simpler version of Fig. 9/Fig. 8 that omits the padding and mask, using only the virtual oracle $\mathfrak{C}_\rho := \sum_{\ell \in \{0,1\}^p} \rho_\ell \mathfrak{C}_\ell$. This base PCS has commit time $2^p T_{C_0}(2^{n-p}) + O(2^n)$, prove time $T_{P_M}(2^{n-p}) + O(2^n)$, and verify time $T_{V_M}(2^{n-p}) + O(k_{0,n-p} \cdot 2^p)$.

5.1 End-to-End Abstract Analysis

Let $P_{\text{MIOP}}, V_{\text{MIOP}}$ denote the base MIOP prove and verify times (including the base PCS costs above), and let $\sigma_R, \sigma_\ell, m', s$ and the sub-protocol codes $C_+, C_\times, C_\times^*, D$ be as in Remark 4.4.

5.1.1 Timing

The zk compilation replaces the base PCS with the zk-multilinear-PCS (Lemma 4.7), then applies the intermediate compilation. Both PCS variants invoke the same base simple MCS (P_M, V_M) on input length 2^{n-p} , so the MCS prove and verify costs cancel. The changes are the PCS commit overhead

$$\Delta_C := (2^p + 1) T_C(2^{n-p} + k) - 2^p T_{C_0}(2^{n-p}),$$

from encoding one additional mask row and k extra padding coordinates per row, and the PCS verify overhead $\Delta_V := O(k_{0,n-p} \cdot k)$, from the enlarged query alphabet (each query to $\mathfrak{C}_\rho$ requires evaluating $C(0, g_1)$ at one coordinate, costing $O(k)$).

The final compilation (Lemma 4.8) thus produces a partially-zk protocol with $P_0 = P_{\text{MIOP}} + \Delta_C$ and $V_0 = V_{\text{MIOP}} + \Delta_V$. Substituting into Remark 4.4:

- **Prove Time:** $P_{\text{MIOP}} + \Delta_C + \sigma_R + \sigma_\ell + 4T_{C_\times}(m') + 2T_{C_\times^*}(m') + 2T_{C_+}(s) + T_D(m') + O(m' + s)$.
- **Verify Time:** $V_{\text{MIOP}} + \Delta_V + \sigma_\ell + T_{C_\times}(m') + T_{C_\times^*}(m') + T_{C_+}(s) + T_D(m') + O(m' + s)$.

5.1.2 Soundness

Let ϵ_0 denote the soundness error of the base MIOP, $\epsilon_{\text{bind,PCS}}$ the binding error of the multilinear PCS (Lemma 4.7), and $\epsilon_{\text{bind,CE}}$ the binding error of the circuit evaluation scheme (Lemma 3.9):

$$\epsilon_{\text{bind,PCS}} = \epsilon_{\text{PG}}(m, |\mathbb{F}|) + \max \{ \epsilon_{\text{PG}}^\bullet + 2^{-\lambda_0}, 2^{-\mu_0 \delta} \}, \quad \epsilon_{\text{bind,CE}} = 3\epsilon_{\text{LB}_\times}^\bullet + \frac{2}{|\mathbb{F}|} + \max \{ \epsilon_+, \epsilon_\times \},$$

where $\epsilon_+, \epsilon_\times$ are the binding errors of the zk-linear-eval (Lemma 3.4) and zk-Hadamard-check (Lemma 3.6), and $\epsilon_{\text{LB}_\times}^\bullet$ is the linear bias of $\text{LB}_\times$.

By Lemma 4.8, the partially-zk protocol has soundness error at most $\epsilon_0 + \epsilon_{\text{bind,PCS}}$. The intermediate compilation (Lemma 4.3) adds $\epsilon_{\text{bind,CE}}$: let h be the witness bound to the circuit evaluation commitment. When $\mathcal{C}'_{s,x,r,v'}(h) = 0$ (probability $\leq \epsilon_0 + \epsilon_{\text{bind,PCS}}$), the circuit evaluation accepts by completeness. When $\mathcal{C}'_{s,x,r,v'}(h) \neq 0$, acceptance requires the circuit evaluation binding to fail (probability $\leq \epsilon_{\text{bind,CE}}$). The total soundness error is at most

$$\epsilon_0 + \epsilon_{\text{bind,PCS}} + \epsilon_{\text{bind,CE}}.$$

5.2 Concrete Instantiation

5.2.1 Code and Proximity Generator Choice

We choose coefficient-to-evaluation Reed-Solomon codes (Example 2.9) for every non-multiplicative code and interpolation Reed-Solomon codes (Example 2.10) for each multiplicative code with C^*, D . All RS codes have rate $1/16$, giving block lengths $m_+ = 16(n' + 6 + \lambda)$ and $m_\times = 16(m' + 2 + \lambda)$ for the circuit evaluation codes, and $2^m = 16(2^{n-p} + k)$ for the PCS code, where λ is the sub-protocol zk padding and $k \geq k_{0,n-p}$ is the PCS zk padding.

As in Example 2.21, the proximity generators in zk-lin-eval and zk-Hadamard-check are **zk-degree- t** for the appropriate t , and the PCS proximity generator is **augmented t -MLE**. For all linear bias generators (LB in zk-Hadamard-check and $\text{LB}_\times$ in zk-circuit-eval, each on vectors of length n), we use the degree- n distribution $(1, \rho, \dots, \rho^{n-1}) \in \mathbb{F}^n$ for ρ uniform over $\mathbb{F}$, giving linear bias $(n-1)/|\mathbb{F}|$ and randomness complexity 1.

5.2.2 Oracle Handling

We realize oracles as follows: Each entry of the oracle is hashed and then all hashes are committed as a Merkle tree whose root $\mathbf{P}$ sends to $\mathbf{V}$. If $\mathbf{V}$ needs to open a proof, it asks $\mathbf{P}$ for the full Merkle path proving the Merkle-tree opening.

For an oracle of length n with entries of size x field elements, generating the tree takes $O(nx)$ prover time (hashing n leaves and $n-1$ internal nodes). Each query requires the prover to send the entry (x field elements) and a Merkle path ($\lceil \log_2 n \rceil$ hash digests). The verifier hashes the entry and walks up the path, costing $O(x + \log n)$ time.

5.2.3 Explicit Overhead

We target b bits of security and substitute the choices of Section 5.2.1 into the abstract analysis of Section 5.1.

Soundness. The concrete binding errors are

$$\begin{aligned} \epsilon_{\text{bind,CE}} &= \frac{3m' + 5}{|\mathbb{F}|} + \max\{\varepsilon_+, \varepsilon_\times\}, & \varepsilon_+ &= \frac{m_+}{|\mathbb{F}|} + \left(\frac{17}{32}\right)^\lambda, & \varepsilon_\times &= \frac{65m' + 64\lambda + 129}{|\mathbb{F}|} + \left(\frac{9}{16}\right)^\lambda, \\ \epsilon_{\text{bind,PCS}} &= \frac{mp}{|\mathbb{F}|} + \max\left\{\frac{p}{|\mathbb{F}|} + 2^{-\lambda_0}, 2^{-15\mu_0/32}\right\}, \end{aligned}$$

where λ_0 is the base MCS binding exponent and μ_0 its IOPP-strength. To achieve $\epsilon_{\text{bind,CE}}, \epsilon_{\text{bind,PCS}} \leq 2^{-b}$ (giving total soundness $\leq \epsilon_0 + 2^{-b+1}$), it suffices to require:

- $\lambda \geq \lceil (b+1)/\log_2(32/17) \rceil$,
- $|\mathbb{F}| \geq \max\{16n' + 3m' + 16\lambda + 101, (m+1)p\} \cdot 2^{b+1}$,
- $\lambda_0 \geq b+1$ and $\mu_0 \geq \lceil 32(b+1)/15 \rceil$.

For $b = 100$: $\lambda \geq 111$, $\lambda_0 \geq 101$, $\mu_0 \geq 216$, and $\log_2 |\mathbb{F}| \geq \max\{\log_2(16n' + 3m' + 1877), \log_2((m+1)p)\} + 101$.

Timing and proof size. Since C_0 and C share the same RS evaluation domain, $T_{C_0}(2^{n-p}) = T_C(2^{n-p}+k) = O(m \cdot 2^m)$, so the PCS commit overhead is one additional RS encoding (for the mask row). The circuit evaluation requires 2 coefficient-to-evaluation encodings with C_+ (domain size m_+) and 6 interpolation encodings with $C_\times$ (domain size $m_\times$, message size $m' + 2$). A coefficient-to-evaluation encoding on domain size N costs one size- N FFT; an interpolation encoding on domain N with message size d costs one size- d inverse FFT plus one size- N forward FFT (Example 2.10). Each size- N FFT costs $\frac{N}{2} \log_2 N$ field multiplications and $N \log_2 N$ additions.

The compiled protocol produces three oracles beyond those of the base MIOP: $\mathfrak{C}_+$ (length m_+ , entry size 2, queried λ times), $\mathfrak{C}_\times$ (length $m_\times$, entry size 5, queried λ times), and the PCS oracle $\mathfrak{C}$ (length 2^m , entry size $2^p + 1$, queried $k_{0,n-p}$ times). Merkle-committing these requires $2^m + m_+ + m_\times$ leaf hashes and the same number of internal-node hashes. In the Merkle-committed protocol, the verifier does not perform full encodings; instead, it evaluates the target codeword polynomials at the queried positions via Horner’s method, costing $\lambda(s + 2m' + 10)$ multiplications for the circuit evaluation codes and $k_{0,n-p} \cdot k$ for the PCS code. Each query also requires hashing the opened entry and verifying a Merkle path.

The ZK overhead beyond the base protocol is:

- **Prove Time:** $\sigma_R + \sigma_\ell$ field multiplications for constraint evaluation; $\frac{1}{2}(m \cdot 2^m + 2m_+ \log_2 m_+ + 6(m' + 2) \log_2(m' + 2) + 6m_\times \log_2 m_\times)$ multiplications for encoding (1 PCS FFT, 2 $\mathfrak{C}_+$ FFTs, 6 $\mathfrak{C}_\times$ iFFT+FFT pairs); $2(2^m + m_+ + m_\times)$ hashes for Merkle commitment.
- **Verify Time:** $\sigma_\ell + \lambda(s + 2m' + 10) + k_{0,n-p} \cdot k$ field multiplications; $k_{0,n-p}(m + 1) + \lambda(\lceil \log_2 m_+ \rceil + \lceil \log_2 m_\times \rceil + 2)$ hashes for Merkle verification.
- **Proof Size:**
 - $k_{0,n-p} \cdot m + \lambda(\lceil \log_2 m_+ \rceil + \lceil \log_2 m_\times \rceil) + 3$ hash digests,
 - $(k_{0,n-p} + 1)(2^p + 1) + 3m' + n' + 11\lambda + k + p + 25$ field elements.

5.3 Experimental Results

Using a proof-of-concept implementation in Rust, we test a standard protocol that (separately) commits two randomly generated multilinear f, g over a 31-bit prime field and proves a Hadamard sumcheck of the form

$$\sum_{x \in \{0,1\}^n} f(x)g(x).$$

For soundness, the sumcheck is run relative to a degree 4 extension of the base field. The sumcheck reduces the claim to evaluation claims on f and g , which are proved with a stacking height of $p = 8$ and using Basefold [ZCF24] for the base PCS (in a future version we plan to change to the WHIR [ACFY25] protocol which has shorter proofs). We target 100 bits of security while using the proximity gap theorem for the unique-decoding regime of [BCI+23]. The IOP is compiled into a succinct argument using the Poseidon2 hash function (see [CY24] for details). We compare the direct non-zk variant of the protocol vs. one compiled to be zk using VEIL.

The benchmarking was done on a MacBook Pro with an M4 processor and 36GB of RAM. We also made some common optimizations for both the zk and non-zk protocol. Many heavy computations—Reed-Solomon encoding, taking RLC’s of the sub-MLE’s f_ℓ , etc — were multithreaded using the Rayon package. Finally, the two final evaluation claims at a common point were batched.

We run each experiment 10 times and take the median for timings. The results are presented in Table 5.1.

Acknowledgments

We thank Matteo Frigo for useful comments.

$\log(\text{trace})$	Prover time (s)			Verifier time (ms)			Proof size (KB)		
	Non-ZK	ZK	Factor	Non-ZK	ZK	Factor	Non-ZK	ZK	Factor
25	3.19	3.32	1.04×	12.49	15.94	1.28×	747	872	1.17×
27	13.00	13.48	1.04×	14.75	18.76	1.27×	868	993	1.14×
29	52.47	54.09	1.03×	17.34	21.13	1.22×	1001	1126	1.12×

Table 5.1: End-to-end ZK overhead for varying trace size.

References

- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed-Solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025.
- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligerio: Lightweight sublinear arguments without a trusted setup. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 2087–2104. ACM, 2017.
- [BCF⁺17] Eli Ben-Sasson, Alessandro Chiesa, Michael A. Forbes, Ariel Gabizon, Michael Riabzev, and Nicholas Spooner. Zero knowledge protocols from succinct constraint detection. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part II*, volume 10678 of *Lecture Notes in Computer Science*, pages 172–206. Springer, 2017.
- [BCI⁺23] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed-Solomon codes. *J. ACM*, 70(5):31:1–31:57, 2023.
- [BCL22] Jonathan Bootle, Alessandro Chiesa, and Siqi Liu. Zero-knowledge IOPs with linear-time prover and polylogarithmic-time verifier. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 275–304. Springer, 2022.
- [BCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I*, pages 103–128, 2019.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography - 14th International Conference, TCC 2016-B, Beijing, China, October 31 - November 3, 2016, Proceedings, Part II*, pages 31–60, 2016.
- [BFLS91] László Babai, Lance Fortnow, Leonid A. Levin, and Mario Szegedy. Checking computations in polylogarithmic time. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, May 5-8, 1991, New Orleans, Louisiana, USA*, pages 21–31, 1991.

- [BFRW25] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. TensorSwitch: Nearly optimal polynomial commitments from tensor codes. *IACR Cryptol. ePrint Arch.*, page 2065, 2025.
- [BGG⁺88] Michael Ben-Or, Oded Goldreich, Shafi Goldwasser, Johan Håstad, Joe Kilian, Silvio Micali, and Phillip Rogaway. Everything provable is provable in zero-knowledge. In Shafi Goldwasser, editor, *Advances in Cryptology - CRYPTO '88, 8th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1988, Proceedings*, volume 403 of *Lecture Notes in Computer Science*, pages 37–56. Springer, 1988.
- [CFS17] Alessandro Chiesa, Michael A. Forbes, and Nicholas Spooner. A zero knowledge sumcheck and its applications. *CoRR*, abs/1704.02086, 2017.
- [CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. *Cryptology ePrint Archive*, Paper 2026/391, 2026.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024.
- [DGR20] Scott E. Decatur, Oded Goldreich, and Dana Ron. A probabilistic error-correcting scheme that provides partial secrecy. In Oded Goldreich, editor, *Computational Complexity and Property Testing - On the Interplay Between Randomness and Computation*, volume 12050 of *Lecture Notes in Computer Science*, pages 1–8. Springer, 2020.
- [Dia25] Benjamin E. Diamond. Zero-knowledge polynomial commitment in binary fields. *IACR Cryptol. ePrint Arch.*, page 1015, 2025.
- [DP24a] Benjamin E Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. *Cryptology ePrint Archive*, 2024.
- [DP24b] Benjamin E. Diamond and Jim Posen. Proximity testing with logarithmic randomness. *IACR Commun. Cryptol.*, 1(1):2, 2024.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.
- [FS24] Matteo Frigo and Abhi Shelat. Anonymous credentials from ECDSA. *IACR Cryptol. ePrint Arch.*, page 2010, 2024.
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. *IACR Cryptol. ePrint Arch.*, page 953, 2019.
- [HK24] Ulrich Haböck and Al Kindi. A note on adding zero-knowledge to STARKs. *IACR Cryptol. ePrint Arch.*, page 1037, 2024.

- [ISVW13] Yuval Ishai, Amit Sahai, Michael Viderman, and Mor Weiss. Zero knowledge LTCs and their applications. In Prasad Raghavendra, Sofya Raskhodnikova, Klaus Jansen, and José D. P. Rolim, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques - 16th International Workshop, APPROX 2013, and 17th International Workshop, RANDOM 2013, Berkeley, CA, USA, August 21-23, 2013. Proceedings*, volume 8096 of *Lecture Notes in Computer Science*, pages 607–622. Springer, 2013.
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, pages 723–732, 1992.
- [KR08] Yael Tauman Kalai and Ran Raz. Interactive PCP. In *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part II - Track B: Logic, Semantics, and Theory of Programming & Track C: Security and Cryptography Foundations*, pages 536–547, 2008.
- [KS25] Darya Kaviani and Srinath Setty. Vega: Low-latency zero-knowledge proofs over existing credentials. Cryptology ePrint Archive, Paper 2025/2094, 2025.
- [Mic00] Silvio Micali. Computationally sound proofs. *SIAM J. Comput.*, 30(4):1253–1298, 2000.
- [NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. *IACR Cryptol. ePrint Arch.*, page 1187, 2025.
- [NN93] Joseph Naor and Moni Naor. Small-bias probability spaces: Efficient constructions and applications. *SIAM J. Comput.*, 22(4):838–856, 1993.
- [RR24] Noga Ron-Zewi and Ron Rothblum. Local proofs approaching the witness length. *J. ACM*, 71(3):18, 2024.
- [RRR21] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. Constant-round interactive proofs for delegating computation. *SIAM J. Comput.*, 50(3), 2021.
- [RW24] Noga Ron-Zewi and Mor Weiss. Zero-knowledge IOPs approaching witness length. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 105–137. Springer, 2024.
- [SP1] SP1 Hypercube. <https://github.com/succinctlabs/sp1>.
- [WTS⁺18] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*, pages 926–943. IEEE Computer Society, 2018.
- [XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 299–328. Springer, 2022.
- [XZZ⁺19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 733–764. Springer, 2019.

- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 138–169. Springer, 2024.